
HYBRID EQUIVALENT PUNCTURED AND QUASI-CYCLIC (HEP-QC)

CHENGJU LI, TIANRUI LIU, CHUNYU GAN, ZHONGYU HUANG,
SHUYING DONG, XINRAN WANG

EAST CHINA NORMAL UNIVERSITY

E-MAIL: cjli@sei.ecnu.edu.cn

JUNE 16, 2026

Contents

1	Introduction	1
2	Preliminaries	2
2.1	Notations	2
2.2	Coding theory	2
2.3	Security assumptions	3
3	Specifications	6
3.1	XOF and Hash functions	6
3.2	Sampling and Generation	6
3.2.1	Vector sampling	6
3.2.2	Matrix and map generation	6
3.3	Vector and matrix multiplications	7
3.4	Concatenated Reed-Muller and Reed-Solomon codes	7
3.4.1	Concatenated codes	7
3.4.2	Reed-Solomon codes	8
3.4.3	Reed-Muller codes	9
3.5	Constructions based on concatenated codes	10
3.6	PKE	10
3.7	KEM	12
4	Parameters and Sizes	14
4.1	Parameter sets	14
4.2	Ciphertext and key sizes	14
5	Performance Analysis	15
5.1	System Configuration	15
5.2	Performance Test Results	15
5.3	Known Answer Test values	15
6	Security	16
6.1	Choices of parameters	16
6.2	Decoding Failure Rate analysis	16
6.2.1	Analysis of the error vector distribution	16
6.2.2	Decoding failure rate analysis	18
6.3	Security Statement and Analysis	18
6.3.1	IND-CPA security	19
6.3.2	IND-CCA2 security	23
6.3.3	Security proof with non uniform randomness sampling	24
6.3.4	Known attacks	25

7	Statement of Algorithm Characteristics	27
7.1	Novelty	27
7.2	Flexibility	27
7.3	Extensibility	27

1. Introduction

HEP-QC is a code-based cryptographic scheme whose security is based on the Equivalent Punctured Code problem with parity constraint (EPC-P) and on variants of the quasi-cyclic syndrome decoding problem. The EPC-P assumption is used to model the indistinguishability of the structured public-key component from a uniformly random matrix, while the 2-DQCSD-P and 3-DQCSD-P assumptions are used to control the distributions appearing in the public key and in the challenge ciphertext. Based on these assumptions, we prove the IND-CPA security of the PKE scheme by two independent reductions and then obtain an IND-CCA2 secure KEM via the salted Fujisaki-Okamoto transformation [24] with implicit rejection.

Compared with the original HQC construction, HEP-QC introduces an additional security layer based on the EPC-P assumption. While the original HQC security proof mainly relies on quasi-cyclic syndrome decoding type assumptions, our construction admits two independent security reductions: one to the EPC-P problem and the other to the 2-DQCSD-P and 3-DQCSD-P problems. Consequently, the structured public-key component is hidden under an additional code-equivalence-type assumption, which strengthens the structural hiding property of the scheme and provides a more robust security justification.

Organization. We recall the required background in Chapter 2 and present the specification of HEP-QC in Chapter 3. Parameter sets are given in Chapter 4. Chapter 5 presents the performance analysis of this scheme. Correctness, security analysis, and known attacks are provided in Chapter 5, respectively.

2. Preliminaries

2.1 Notations

Let $\mathbb{Z}$ denote the ring of integers and $\mathbb{F}_2$ the binary finite field. Let $\mathcal{V}$ be a vector space of dimension n over $\mathbb{F}_2$ for some positive integer n . The elements of $\mathcal{V}$ may be viewed interchangeably as row vectors or as polynomials in the ring $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$. Vectors and polynomials are denoted by lowercase bold letters, and matrices by uppercase bold letters.

A prime integer n is called *primitive* if the polynomial $(X^n - 1)/(X - 1)$ is irreducible in $\mathcal{R}$. For $\mathbf{u}, \mathbf{v} \in \mathcal{V}$, their product in $\mathcal{R}$ is defined as $\mathbf{w} = \mathbf{u} \cdot \mathbf{v} \in \mathcal{V}$ with

$$w_k = \sum_{i+j \equiv k \pmod{n}} u_i \cdot v_j \quad \forall k \in \{0, 1, \dots, n-1\}.$$

Given $\mathbf{v} \in \mathbb{F}_2^n$, let $\text{rot}(\mathbf{v})$ denote the circulant matrix induced by $\mathbf{v}$:

$$\text{rot}(\mathbf{v}) = \begin{pmatrix} v_0 & v_{n-1} & \cdots & v_1 \\ v_1 & v_0 & \cdots & v_2 \\ \vdots & \vdots & \ddots & \vdots \\ v_{n-1} & v_{n-2} & \cdots & v_0 \end{pmatrix} \in \mathbb{F}_2^{n \times n}.$$

The product of two elements $\mathbf{u}, \mathbf{v} \in \mathcal{R}$ can be expressed using the $\text{rot}(\cdot)$ operator as

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{u} \times \text{rot}(\mathbf{v})^\top = (\text{rot}(\mathbf{u}) \times \mathbf{v}^\top)^\top = \mathbf{v} \times \text{rot}(\mathbf{u})^\top = \mathbf{v} \cdot \mathbf{u}.$$

Denote by $\omega(\cdot)$ the Hamming weight of a vector, i.e., the number of its nonzero coordinates. For a positive integer ω , let $\mathcal{R}_\omega := \{\mathbf{v} \in \mathcal{R} \mid \omega(\mathbf{v}) = \omega\}$ be the set of vectors of Hamming weight ω . The Hamming distance $d(\mathbf{u}, \mathbf{v})$ of two vectors $\mathbf{u}$ and $\mathbf{v}$ is defined to be the number of coordinates in which $\mathbf{u}$ and $\mathbf{v}$ differ.

2.2 Coding theory

In the following, we provide basic definitions and properties about coding theory and refer the reader to [25] for more details.

Definition 2.2.1 (Linear Code). *A linear code $\mathcal{C}$ of length n and dimension k is a subspace of $\mathcal{R}$ of dimension k .*

Definition 2.2.2 (Generator Matrix). *Given an $[n, k]$ code $\mathcal{C}$, a matrix $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ is a generator matrix for $\mathcal{C}$ if:*

$$\mathcal{C} = \{\mathbf{m}\mathbf{G} \mid \forall \mathbf{m} \in \mathbb{F}_2^k\}.$$

Definition 2.2.3 (Parity-Check Matrix). *Given an $[n, k]$ code $\mathcal{C}$, a matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ is a parity-check matrix for $\mathcal{C}$ if $\mathbf{H}$ is a generator matrix of the dual code $\mathcal{C}^\perp$:*

$$\mathcal{C} = \{\mathbf{v} \in \mathbb{F}_2^n \mid \mathbf{H}\mathbf{v}^\top = 0\}$$

or equivalently

$$\mathcal{C}^\perp = \{\mathbf{u}\mathbf{H} \mid \forall \mathbf{u} \in \mathbb{F}_2^{n-k}\}.$$

Definition 2.2.4 (Syndrome). Let $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ be a parity-check matrix of some $[n, k]$ code $\mathcal{C}$, and $\mathbf{v} \in \mathbb{F}_2^n$ be a codeword. The syndrome of $\mathbf{v}$ is $\mathbf{H}\mathbf{v}^\top$, and one has $\mathbf{v} \in \mathcal{C} \iff \mathbf{H}\mathbf{v}^\top = 0$.

Definition 2.2.5 (Minimum Distance). The minimum distance of an $[n, k]$ linear code $\mathcal{C}$ is:

$$d = \min\{d(\mathbf{u}, \mathbf{v}) \mid \mathbf{u}, \mathbf{v} \in \mathcal{C}, \mathbf{u} \neq \mathbf{v}\}.$$

A code of length n and dimension k with minimum distance d is denoted as an $[n, k, d]$ code.

Definition 2.2.6 (Quasi-Cyclic Codes). An $[sn, k, d]$ linear code $\mathcal{C}$ is Quasi-Cyclic (QC) of index s if for any $\mathbf{c} = (\mathbf{c}_0, \dots, \mathbf{c}_{s-1}) \in \mathcal{C}$, the vector obtained after applying a simultaneous circular shift to every block $\mathbf{c}_0, \dots, \mathbf{c}_{s-1}$ is also a codeword. More formally, by considering each block $\mathbf{c}_i$ as a polynomial in $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$, the code $\mathcal{C}$ is QC of index s if for any $\mathbf{c} = (\mathbf{c}_0, \dots, \mathbf{c}_{s-1}) \in \mathcal{C}$ it holds that $(X \cdot \mathbf{c}_0, \dots, X \cdot \mathbf{c}_{s-1}) \in \mathcal{C}$.

Definition 2.2.7 (Systematic Quasi-Cyclic Codes). A systematic Quasi-Cyclic $[sn, n]$ code of index s and rate $1/s$ is a quasi-cyclic code with an $(s-1)n \times sn$ parity-check matrix of the form:

$$\mathbf{H} = \begin{bmatrix} \mathbf{I}_n & 0 & \cdots & 0 & \mathbf{A}_0 \\ 0 & \mathbf{I}_n & & & \mathbf{A}_1 \\ & & \ddots & & \vdots \\ 0 & \cdots & 0 & \mathbf{I}_n & \mathbf{A}_{s-2} \end{bmatrix},$$

where $\mathbf{A}_0, \dots, \mathbf{A}_{s-2}$ are circulant $n \times n$ matrices.

2.3 Security assumptions

In this section we describe difficult problems which are relevant for HEP-QC and discuss their complexity. In the following let F denote a finite field. We introduce the definition of punctured code at first.

Definition 2.3.1. Let $\mathcal{C}$ be a code of length n and $S \subset \{1, \dots, n\} := N$. Denote the code $\mathcal{C}_S$ which is obtained by deleting all the coordinates of $\mathcal{C}$ in $N \setminus S$. $\mathcal{C}_S$ is called punctured code of $\mathcal{C}$ in $N \setminus S$.

Consider the following problem: let $\mathcal{C}$ be a code of length n and $\mathcal{D}$ a code of length $m, m \leq n$. Does there exist a subset $S \subset \{1, \dots, n\}$ such that $\mathcal{C}_S$ and $\mathcal{D}$ are equal? The problem can be rewritten as the following Punctured Code (PC) Problem:

Definition 2.3.2 (Punctured Code Problem, PC Problem). Let $\mathbf{M}$ be a $k \times n$ matrix, $\mathbf{H}$ a $k \times m$ matrix, $m \leq n$, both over F . Does there exist a non-singular matrix $\mathbf{T}$ and a subsets $S \subset \{1, 2, \dots, n\}$ with $|S| = m$ such that $(\mathbf{T}\mathbf{M})_S = \mathbf{T}(\mathbf{M})_S = \mathbf{H}$?

PC problem has been proven NP-complete [41]. There is a way to strict PC if we consider the code equivalent problem:

Definition 2.3.3. Let $\mathcal{C}, \mathcal{D}$ be codes over the same field and of same length n . $\mathcal{C}, \mathcal{D}$ are called equivalent, if there is a permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ such that

$$(c_1, \dots, c_n) \in \mathcal{C} \iff (c_{\pi(1)}, \dots, c_{\pi(n)}) \in \mathcal{D}.$$

Definition 2.3.4. Let $\mathbf{M}$ be a $k \times n$ matrix over F with columns $c_1, \dots, c_n$ and $\tau : \{1, \dots, m\} \rightarrow \{1, \dots, n\}$ an injection ($m \leq n$). Denote $\mathbf{M}_\tau$ the $k \times m$ matrix consisting of the columns $c_{\tau(1)}, \dots, c_{\tau(m)}$ in this order.

For example, we allow the code $\mathcal{C}_S$ to be equivalent to $\mathcal{D}$, then there is a harder problem exist: let $\mathcal{C}$ be a code of length n and $\mathcal{D}$ a code of length $m, m \leq n$. Does there exist a subset $S \subset \{1, \dots, n\}$ such that $\mathcal{C}_S$ and $\mathcal{D}$ are equivalent? The problem can be rewritten as the following Equivalent Punctured Code (EPC) distribution and problem:

Definition 2.3.5 (EPC Distribution). *Let k, m and n be positive integers with $m \leq n$ and F a field. The Equivalent Punctured Code Distribution $EPC(k, m, n)$ samples $\mathbf{M} \leftarrow \$F^{k \times n}$, a non-singular matrix $\mathbf{T} \leftarrow \$F^{k \times k}$ and an injection $\tau : \{1, \dots, m\} \rightarrow \{1, \dots, n\}$, computes $\mathbf{H} = \mathbf{T}\mathbf{M}_\tau$ and outputs $(\mathbf{M}, \mathbf{H})$.*

Definition 2.3.6 (EPC Problem). *Given a $k \times n$ matrix $\mathbf{M}$ over F and a $k \times m$ matrix $\mathbf{H}$ over F with $m \leq n$, does there exist a non-singular $k \times k$ matrix $\mathbf{T}$ and an injective map $\tau : \{1, \dots, m\} \rightarrow \{1, \dots, n\}$ with $(\mathbf{T}\mathbf{M})_\tau = \mathbf{T}\mathbf{M}_\tau = \mathbf{H}$?*

The 3-Dimensional Matching (3DM) problem can be reduced to EPC, then EPC problem has been proven NP-complete [41, Proposition 1]. Our algorithm considers the EPC problem: Two matrices $\mathbf{G} \in \mathbb{F}_2^{k \times n}, \mathbf{G}' \in \mathbb{F}_2^{k \times n'}, n < n'$ satisfy the relation $(\mathbf{T}\mathbf{G}')_\tau = \mathbf{G}$ for an injection $\tau : \{1, \dots, n'\} \rightarrow \{1, \dots, n\}$ and a non-singular $k \times k$ matrix $\mathbf{T}$. It is difficult to find the private key $\mathbf{T}$ and τ since the matrix $\mathbf{G}'$ appears to be a random matrix, just like the Goppa code used in McEliece scheme.

Our scheme also depends on some difficult problems which are relevant for HQC scheme. All the problems are variants of the decoding problem, we begin to introduce problems that are relevant to Syndrome Decoding (SD) problem.

Definition 2.3.7 (SD Distribution). *Let n, k and ω be positive integers. The Syndrome Decoding Distribution $SD(n, k, \omega)$ samples $\mathbf{H} \leftarrow \$\mathbb{F}_2^{(n-k) \times n}$ and $\mathbf{x} \leftarrow \$\mathbb{F}_2^n$ such that $\text{wt}(\mathbf{x}) = \omega$, computes $\mathbf{y}^\top = \mathbf{H}\mathbf{x}^\top$ and outputs $(\mathbf{H}, \mathbf{y})$.*

Definition 2.3.8 (SD Problem). *Let n, k and ω be positive integers. Given $(\mathbf{H}, \mathbf{y}) \in \mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k}$ from the $SD(n, k, \omega)$ distribution, the Syndrome Decoding Problem $SD(n, k, \omega)$ asks to find $\mathbf{x} \in \mathbb{F}_2^n$ such that $\mathbf{y}^\top = \mathbf{H}\mathbf{x}^\top$ and $\text{wt}(\mathbf{x}) = \omega$.*

Definition 2.3.9 (DSD Problem). *Let n, k and ω be positive integers. Given $(\mathbf{H}, \mathbf{y}) \in \mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k}$, the Decisional Syndrome Decoding Problem $DSD(n, k, \omega)$ asks to decide with non-negligible advantage whether $(\mathbf{H}, \mathbf{y})$ came from the uniform distribution over $\mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k}$ or the $SD(n, k, \omega)$ distribution.*

Our scheme depends on the matrix $\mathbf{G}$ to encode message $\mathbf{m}$, it is difficult to find $\mathbf{m}$ since the original matrix $\mathbf{G}'$ performances like a random matrix, and it turns on a DSD Problem. Since our scheme also uses QC-codes, then we need to describe the DSD problem under the QC setting.

Definition 2.3.10 (s -QCSD Distribution). *Let n, s and ω be positive integers. The s -Quasi-Cyclic Syndrome Decoding Distribution $s\text{-QCSD}(n, \omega)$ samples a parity-check matrix $\mathbf{H} \leftarrow \$\mathbb{F}_2^{(sn-n) \times sn}$ of a systematic Quasi Cyclic code $\mathcal{C}$ of index s and rate $\frac{1}{s}$ and a vector $\mathbf{x} = (\mathbf{x}_0, \dots, \mathbf{x}_{s-1}) \leftarrow \$\mathbb{F}_2^{sn}$ such that $\text{wt}(\mathbf{x}_i) = \omega \forall i \in [0, s-1]$, computes $\mathbf{y}^\top = \mathbf{H}\mathbf{x}^\top$ and outputs $(\mathbf{H}, \mathbf{y})$.*

Definition 2.3.11 (s -DQCSD Problem). *Let n, s and ω be positive integers. Given $(\mathbf{H}, \mathbf{y}) \in \mathbb{F}_2^{(sn-n) \times sn} \times \mathbb{F}_2^{sn-n}$, the Decisional s -Quasi-Cyclic Syndrome Decoding Problem $s\text{-DQCSD}(n, \omega)$ asks to decide with non-negligible advantage whether $(\mathbf{H}, \mathbf{y})$ came from the $s\text{-QCSD}(n, \omega)$ distribution or the uniform distribution over $\mathbb{F}_2^{(sn-n) \times sn} \times \mathbb{F}_2^{sn-n}$.*

Although there is no universally accepted complexity result for QC codes, decoding them is widely considered a difficult problem. While some general attack methods exploit the cyclic structure of QC codes exist, these attacks have a minimal impact on the problem complexity. To avoid irrelevant distinctions, an additional condition regarding the parity of the syndrome was added to the problem. For

$b_1 \in \{0, 1\}$, let $\mathbb{F}_{2,b_1}^n := \{\mathbf{h} \in \mathbb{F}_2^n : \mathbf{h}(1) = b_1 \bmod 2\}$ denote the finite set that consists of all binary vectors of length n and parity b_1 , define the following two finite sets:

$$\mathbb{F}_{2,b_1}^{n \times 2n} = \{\mathbf{H} = (\mathbf{I}_n \text{ rot } (\mathbf{h})) \in \mathbb{F}_2^{n \times 2n} \text{ s.t. } \mathbf{h} \in \mathbb{F}_{2,b_1}^n\},$$

$$\mathbb{F}_{2,b_1,b_2}^{2n \times 3n} = \left\{ \mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}_1) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{h}_2) \end{pmatrix} \in \mathbb{F}_2^{2n \times 3n} \text{ s.t. } \mathbf{h}_1 \in \mathbb{F}_{2,b_1}^n \text{ and } \mathbf{h}_2 \in \mathbb{F}_{2,b_2}^n \right\}.$$

Definition 2.3.12 (2-QCSD Distribution). *Let n, ω and b_1 be positive integers and let $b_2 = \omega + b_1 \times \omega \bmod 2$. The 2-Quasi-Cyclic Syndrome Decoding with Parity Distribution 2-QCSD(n, ω, b_1, b_2) samples $\mathbf{H} \in \mathbb{F}_{2,b_1}^{n \times 2n}$ and $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2) \leftarrow \$\mathbb{F}_2^{2n}$ such that $\text{wt}(\mathbf{x}_1) = \text{wt}(\mathbf{x}_2) = \omega$, compute $\mathbf{y}^\top = \mathbf{H}\mathbf{x}^\top$ and outputs $(\mathbf{H}, \mathbf{y}) \in \mathbb{F}_{2,b_1}^{n \times 2n} \times \mathbb{F}_{2,b_2}^n$.*

Definition 2.3.13 (2-QCSD-P Problem). *Let n, ω and b_1 be positive integers and let $b_2 = \omega + b_1 \times \omega \bmod 2$. Given $(\mathbf{H}, \mathbf{y}) \in \mathbb{F}_{2,b_1}^{n \times 2n} \times \mathbb{F}_{2,b_2}^n$, the Decisional 2-Quasi-Cyclic Syndrome Decoding with Parity Distribution 2-QCSD-P(n, ω, b_1, b_2) asks to decide with non-negligible advantage whether $(\mathbf{H}, \mathbf{y})$ came from the 2-QCSD(n, ω, b_1, b_2) Distribution or the uniform distribution over $\mathbb{F}_{2,b_1}^{n \times 2n} \times \mathbb{F}_{2,b_2}^n$.*

Similarly, we define the following 3-QCSD Distribution and 3-QCSD-P Problem, which will be used in our security proof.

Definition 2.3.14 (3-QCSD Distribution). *Let n, ω and b_1, b_2 be positive integers and let $b_3 = \omega + b_1 \times \omega \bmod 2$. The 3-Quasi-Cyclic Syndrome Decoding with Parity Distribution 3-QCSD(n, ω, b_1, b_2, b_3) samples $\mathbf{H} \in \mathbb{F}_{2,b_1,b_2}^{2n \times 3n}$ and $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3) \leftarrow \$\mathbb{F}_2^{3n}$ such that $\text{wt}(\mathbf{x}_1) = \text{wt}(\mathbf{x}_2) = \text{wt}(\mathbf{x}_3) = \omega$, computes $(\mathbf{y}_1, \mathbf{y}_2)^\top = \mathbf{H}\mathbf{x}^\top$ and outputs $(\mathbf{H}, (\mathbf{y}_1, \mathbf{y}_2)) \in \mathbb{F}_{2,b_1,b_2}^{2n \times 3n} \times (\mathbb{F}_{2,b_3}^n \times \mathbb{F}_2^n)$.*

Definition 2.3.15 (3-QCSD-P Problem). *Let n, ω and b_1, b_2 be positive integers and let $b_3 = \omega + b_1 \times \omega \bmod 2$. Given $(\mathbf{H}, (\mathbf{y}_1, \mathbf{y}_2)) \in \mathbb{F}_{2,b_1,b_2}^{2n \times 3n} \times (\mathbb{F}_{2,b_3}^n \times \mathbb{F}_2^n)$, the Decisional 3-Quasi-Cyclic Syndrome Decoding with Parity Distribution 3-QCSD-P(n, ω, b_1, b_2, b_3) asks to decide with non-negligible advantage whether $(\mathbf{H}, (\mathbf{y}_1, \mathbf{y}_2))$ came from the 3-QCSD(n, ω, b_1, b_2, b_3) Distribution or the uniform distribution over $\mathbb{F}_{2,b_1,b_2}^{2n \times 3n} \times (\mathbb{F}_{2,b_3}^n \times \mathbb{F}_2^n)$.*

3. Specifications

3.1 XOF and Hash functions

Following the algorithm submission requirements, our implementation utilizes the functions from the `API_PKC.zip` file. These functions are used solely for correctness verification and preliminary performance testing. In subsequent rounds of the evaluation, they will be replaced with cryptographically secure hash functions and extendable-output functions (XOFs) that meet the required security levels.

XOF. An extensible output function (XOF) constructed using KDF-SM3 to generate an output of specified length.

Hash functions. Our scheme employs the SM3 hash function, as well as a hash function that is built using SM3 and HMAC-SM3. This hash function is capable of generating hash values of lengths 512, 768, or 1024 bits.

3.2 Sampling and Generation

3.2.1 Vector sampling

Sampling random vectors. Sampling random vectors from $\mathbb{F}_2^k$ and $\mathbb{F}_2^n$ uses **SampleVect** and uniform sampling. For **SampleVect**, the random bits are derived from a seed using **XOF.GetBytes**, and meanwhile unused bits within the vector representation are set to zero.

Sampling random fixed weight vectors. Random fixed weight vectors are sampled from $\mathbb{F}_2^n$ using either uniform sampling referring to **SampleFixedWeightVect_s** or a slightly biased sampling namely **SampleFixedWeightVect**. **SampleFixedWeightVect_s** relies on rejection sampling but **SampleFixedWeightVect** does not. It is shown that the biased sampling used in the encryption procedure of PKE hardly affects the security of the scheme of Proposition 6.3.5, while it significantly enhances the efficiency of the encryption procedure.

3.2.2 Matrix and map generation

Generating random non-singular square matrices. Random non-singular square matrices are generated from $GL(k, \mathbb{F}_2)$ using **GenerateMat**. Given an input seed, **GenerateMat** deterministically derives a candidate square matrix together with an updated seed. The matrix is then tested for non-singularity. If it is invertible, it is returned as the output; otherwise, the procedure is repeated using the updated seed until a non-singular matrix is obtained.

Generating a Random Permutation. Let $\tau : \{1, \dots, n'\} \rightarrow \{1, \dots, n'\}$ be a permutation, where $n' = 64 \lceil n/64 \rceil$. Let **P** denote the permutation matrix corresponding to τ ; we refer to **P** as permutation for short. The permutation **P** is generated using the procedure **GeneratePerm_s**. Concretely, pseudorandom bits are deterministically derived from a given seed via (**XOF.GetBytes**), and are used to sample integers in the set $\{1, \dots, n'\}$. To ensure that τ defines a valid bijection, rejection sampling is employed to eliminate repeated or out-of-range values during the construction process.

3.3 Vector and matrix multiplications

Truncate and Expand vectors. Let $\mathbf{z} = \{z_1, z_2, \dots, z_r\}$ be a vector of length r , where $r \in \mathbb{N}$. We now define two operations on this vector:

1. Given a non-negative integer l such that $l \leq r$, we define the operation $\text{TruncateVect}(\mathbf{z}, l)$ as follows:

$$\text{TruncateVect}(\mathbf{z}, l) = (z_1, z_2, \dots, z_{r-l}).$$

2. Given a non-negative integer l such that $l \leq r$, we define the operation $\text{ExpandVect}(\mathbf{z}, l)$ as follows:

$$\text{ExpandVect}(\mathbf{z}, l) = (z_1, z_2, \dots, z_r, 0_l).$$

where 0_l represents a vector of length l consisting entirely of zeros.

Vector multiplications. Vector multiplication include two parts: multiplication process and reduction process. To define vector multiplication, we consider two vectors of length N as polynomials of degree $N - 1$, where each vector represents the coefficients of the respective polynomial. When multiplying two polynomials of length N , the result is a polynomial of degree $2N - 2$, and therefore, the resultant vector will have $2N - 1$ coefficients. After obtaining the product of these two polynomials, the next step is to reduce the result modulo $x^N - 1$. This operation effectively ensures that the resulting polynomial stays within the desired degree of $N - 1$, making it a valid polynomial representation in the given ring of polynomials modulo $x^N - 1$. For efficiency, we can use the Karatsuba algorithm to speed up the multiplication process.

Matrix multiplications. Apart from vector multiplications, all other multiplications are standard matrix multiplications. To distinguish between these two types of operations, we adopt the following notational convention: the multiplication symbol $\cdot$ is omitted in the case of matrix multiplications, whereas it is explicitly retained to denote vector multiplications.

3.4 Concatenated Reed-Muller and Reed-Solomon codes

3.4.1 Concatenated codes

Definition 3.4.1 (concatenated codes). *A concatenated code consists of an external code $[n_e, k_e, d_e]$ over $\mathbb{F}_q$ and an internal code $[n_i, k_i, d_i]$ over $\mathbb{F}_2$, with $q = 2^{k_i}$. We use a bijection between elements of $\mathbb{F}_q$ and the words of the internal code to obtain a transformation:*

$$\mathbb{F}_q^{n_e} \rightarrow \mathbb{F}_2^N$$

where $N = n_e n_i$. The external code is thus transformed into a binary code of parameters

$$[N = n_e n_i, K = k_e k_i, D \geq d_e d_i].$$

For the external code, we use a Reed-Solomon code of dimension 32 over $\mathbb{F}_{256}$. For the internal code, we use the Reed-Muller code $[128, 8, 64]$ that we duplicate 3 or 5 times (i.e. duplicating each bit to obtain codes of parameters $[384, 8, 192]$ and $[640, 8, 320]$). We perform maximum likelihood decoding on the internal code. Doing that, we obtain a vector of $\mathbb{F}_q^{n_e}$ that is then decoded using an algebraic decoder for the Reed-Solomon code.

3.4.2 Reed-Solomon codes

Let p be a prime number and q is any power of p . Following [26], a Reed-Solomon code $\text{RS}[n, k, d_{\min}]$ with symbols in $\mathbb{F}_q$ has the following parameters: block length $n = q - 1$; number of parity-check digits $n - k = 2\delta$, with δ , the correcting capacity of the code and k the number of information bits; minimum distance $d_{\min} = 2\delta + 1$.

Let α be a primitive element in $\mathbb{F}_{2^m}$, the generator polynomial $g(x)$ of the $\text{RS}[n, k, \delta]$ code is given by:

$$g(x) = (x + \alpha)(x + \alpha^2) \cdots (x + \alpha^{2\delta}).$$

Shortened Reed-Solomon codes. Depending on the parameters, shortened Reed-Solomon codes are constructed such that k is equal to 16, 24 or 32 from the original RS codes [26]. The shortened codes are obtained by subtracting 209 from the parameters n and k of the code RS-1, subtracting a same number from the parameters n and k of the original RS codes to reach the dimension of the shortened RS codes.

Generator polynomials. For HEP-QC, we are working in $\mathbb{F}_{2^m}$ with $m = 8$. To do so, we use the primitive polynomial $1 + \alpha^2 + \alpha^3 + \alpha^4 + \alpha^8$ of degree 8 to build this field [26]. The generator polynomials of the shortened Reed-Solomon codes are identical to those of the original Reed-Solomon codes respectively. The generator polynomials of the shortened Reed-Solomon codes have been precomputed. One can check the generator polynomials provided in the file `parameters.h` use the functions provided in the file `reed_solomon.h` to reconstruct these generator polynomials.

Encoding shortened Reed-Solomon codes. We describe the encoding procedure for Reed-Solomon (RS) codes, which naturally extends to shortened RS codes. Let $u = (u_0, \dots, u_{k-1}) \in \mathbb{F}_q^k$ be the message vector, and let $u(x) = \sum_{i=0}^{k-1} u_i x^i$ denote its associated polynomial. Let $g(x)$ be the generator polynomial of the code.

We adopt a systematic encoding scheme, in which the k message symbols explicitly appear in the codeword. In this setting, the codeword polynomial is expressed as $c(x) = x^{n-k}u(x) + b(x)$, where $b(x)$ is the remainder obtained when dividing $x^{n-k}u(x)$ by $g(x)$. By construction, $b(x)$ has degree strictly less than $n - k$ and corresponds to the parity part of the codeword.

The encoding process can be summarized as follows: first, shift the message polynomial by multiplying it by x^{n-k} ; next, compute the remainder upon division by $g(x)$; finally, add this remainder to obtain the codeword polynomial. This formulation directly applies to shortened RS codes by embedding the shortened message into the full-length structure.

Decoding shortened Reed-Solomon codes. The decoding of shortened Reed-Solomon (RS) codes is performed by relying on the decoding procedure of the corresponding full-length RS code. For clarity, we describe the classical decoding process. Let $v(x)$ be the transmitted codeword and $r(x)$ the received polynomial, possibly corrupted by errors. Define the error polynomial $e(x) = r(x) - v(x)$, and assume that $e(x)$ has t nonzero terms located at positions $j_1, \dots, j_t$.

The decoding begins by computing the syndromes $S_i = r(\alpha^i)$ for $i = 1, \dots, 2\delta$, where α is a primitive element of $\mathbb{F}_{2^m}$. Since valid codewords vanish at these evaluation points, the syndromes depend only on the error polynomial. Introducing the error-location numbers $\beta * \ell = \alpha^{j_\ell}$, the syndromes can be expressed as a system of equations involving the unknown error locations and magnitudes.

To recover the error locations, one defines the error-location polynomial

$$\sigma(x) = \prod_{\ell=1}^t (1 + \beta_\ell x) = 1 + \sigma_1 x + \cdots + \sigma_t x^t,$$

whose roots are given by $\beta_1^{-1}, \dots, \beta_t^{-1}$. The coefficients of $\sigma(x)$ can be determined from the syndromes using an algebraic decoding algorithm such as the Berlekamp–Massey algorithm [26].

Once the error-location polynomial is known, the error positions are obtained by finding its roots and taking their inverses. The corresponding error values are then computed using an auxiliary polynomial

$$Z(x) = 1 + (S_1 + \sigma_1)x + (S_2 + \sigma_1 S_1 + \sigma_2)x^2 + \dots + (S_t + \sigma_1 S_{t-1} + \dots + \sigma_t)x^t.$$

For each error location β_ℓ , the error magnitude is given [3] by

$$e_{j_\ell} = \frac{Z(\beta_\ell^{-1})}{\prod_{i=1, i \neq \ell}^t (1 + \beta_i \beta_\ell^{-1})}.$$

Finally, the original codeword is recovered by subtracting the error polynomial from the received polynomial, i.e., $v(x) = r(x) - e(x)$.

The overall decoding procedure can be summarized as follows:

1. Compute the 2δ syndromes by evaluating the received polynomial at α^i .
2. Determine the error-location polynomial $\sigma(x)$ from the syndromes.
3. Find the error locations by computing the roots of $\sigma(x)$ and taking their inverses. We implement this step with additive Fast Fourier Transform [19].
4. Construct the auxiliary polynomial $Z(x)$.
5. Compute the error magnitudes.
6. Correct the received polynomial to recover the transmitted codeword.

3.4.3 Reed-Muller codes

For any positive integers m and r with $0 \leq r \leq m$, there exists a binary r^{th} order ReedMuller code denoted by $\text{RM}(r, m)$ with the following parameters: code length $n = 2^m$; dimension $k = \sum_{i=0}^r \binom{m}{i}$; minimum distance $d_{\min} = 2^{m-r}$. In HEP-QC, we use the duplicated Reed-Muller codes. In particular, we are using first-order Reed-Muller code denoted $\text{RM}(1,7)$ which is the binary code [128,8,64]. In Chapter 14 of [30], it explicitly describes the encoding and decoding of the Reed-Muller code of order 1.

Encoding Duplicated Reed-Muller codes. Following [30], the encoding is done in classical way by using a matrix vector multiplication. The codeword is then duplicated depending on the multiplicity of the duplicated Reed-Muller code.

Decoding Duplicated Reed-Muller codes. Following [30], the decoding of duplicated Reed-Muller codes is done in three steps:

1. Decoding the duplicated Reed-Muller code to the Reed-Muller code depending on maximum likelihood decoding. Since the multiplicity of the duplicated code is odd, the result of the maximum likelihood decoding is unique.
2. Compute the Hadamard transform which is the first phase of the Green machine.
3. Compute the location of the highest value on the output of the previous step. This is the second phase of the Green machine. When the peak is positive we add the all-one-vector and if there are two identical peaks, the peak with smallest value in the lowest 7 bits it taken.

3.5 Constructions based on concatenated codes

Expanding concatenated codes to linear codes. To construct a linear expansion of a code, let $\mathcal{C} \subseteq \mathbb{F}^n$ be a linear code with generator matrix $\mathbf{G} \in \mathbb{F}^{k \times n}$. To extend the code to length $n' = 64 \lceil n/64 \rceil$, choose a matrix $\mathbf{A} \in \mathbb{F}^{n \times (n' - n)}$, typically sampled uniformly at random. The extended code $\mathcal{C}' \subseteq \mathbb{F}^{n'}$ is then defined by appending a linear transformation of each codeword:

$$\bar{\mathbf{c}} = (\mathbf{c} \mid \mathbf{c}\mathbf{A}), \quad \text{for all } \mathbf{c} \in \mathcal{C}.$$

Equivalently, the generator matrix of the extended code is obtained by augmenting $\mathbf{G}$ as

$$\bar{\mathbf{G}} = [\mathbf{G} \mid \mathbf{G}\mathbf{A}] \in \mathbb{F}^{k \times n'}.$$

In this construction, the appended part $\mathbf{c}\mathbf{A}$ depends linearly on the original codeword, ensuring that $\bar{\mathcal{C}}$ remains a linear code. If $\mathbf{A}$ is chosen uniformly at random, then the matrix $\mathbf{G}\mathbf{A}$ is uniformly random in $\mathbb{F}^{k \times (n' - n)}$, so the extension can be viewed as adding random linear redundancy while preserving linearity. Hence, the matrix $\mathbf{G}\mathbf{A}$ can be generated by using **SampleVect**_{\$} to sample vectors in $\mathbb{F}_q^k$ for $(n' - n)$ times, as well as $\bar{\mathbf{G}}$. We refer to the generation of expanded concatenated codes as **ExpandMat**_{\$}($\mathbf{G}, n' - n$).

Permutation to the coordinates of the expanded codes. Essentially, expanded concatenated codes are constructed by appending additional randomly generated coordinates to the original concatenated codes, with the purpose of increasing the difficulty of decoding. These extra coordinates do not carry structured redundancy in the same way as the original code components, and thus act as a form of obfuscation that interferes with standard decoding strategies.

To reach this purpose, the permutation generated by **GeneratePerm**_{\$} is applied to the extended codeword to uniformly shuffle all coordinates. As a result, the positions of the randomly added coordinates become indistinguishable from those originating from the underlying concatenated code. This mixing effect prevents an adversary from reliably identifying and isolating the appended random part.

Consequently, any attempt to decode the expanded code by simply puncturing a subset of coordinates is rendered ineffective, since there is no efficient way to determine which positions correspond to the random expansion. The combination of random expansion and permutation therefore ensures that the structure of the original concatenated codes are well hidden, significantly increasing the complexity of decoding without knowledge of the secret permutation.

3.6 PKE

HEP-QC uses an undecodable $[n', k]$ code $\mathcal{C}'$ and a random double-circulant $[2n, n]$ code with parity-check matrix $(\mathbf{I}_n \mid \text{rot}(\mathbf{h}))$ to encode, while a decodable $[n_1 n_2, k]$ code $\mathcal{C}$ related to $\mathcal{C}'$ to decode. The code $\mathcal{C}$ is instantiated using concatenated Reed-Muller and Reed-Solomon codes which are described in Section 3.4. The code $\mathcal{C}$ is instantiated using the expansion and the permutation described in Section 3.5. All computations are made in $\mathbb{F}_2^n$ or $\mathbb{F}_2^{n'}$, and all the operations including truncation and expansion are described in Section 3.3 and 3.5.

Correctness. The correctness of HEP-QC relies on the decoding capability of the code $\mathcal{C}$. Specifically, $\mathcal{C}$ correctly decodes whenever:

$$\omega(\text{TruncateVect}(\text{ExpandVect}(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}, n' - n) \mathbf{P}, n' - n)) \leq \Delta$$

An analysis of the distribution of the error vector $\mathbf{e}' = \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}$ and the resulting decoding

failure rate of the scheme is provided in Section 6.3.3.

Algorithm 1 HEP-QC-PKE.KeyGen(seed_{PKE})

Require: Random seed seed_{PKE}.

Ensure: Public key pk_{PKE}, secret key sk_{PKE}.

```

// Compute pk and sk seeds
1: (seedPKE.sk, seedPKE.pk) ← H(seedPKE)

// Compute secret key sk
2: ctxPKE.sk ← XOF.Init(seedPKE.sk)
3: (ctxPKE.sk, y) ← SampleFixedWeightVect§(ctxPKE.sk,  $\mathcal{R}_\omega$ )
4: (ctxPKE.sk, x) ← SampleFixedWeightVect§(ctxPKE.sk,  $\mathcal{R}_\omega$ )
5: (ctxPKE.sk, P) ← GeneratePerm§(ctxPKE.sk,  $\mathcal{R}$ )
6: (ctxPKE.sk, T) ← GenerateMat§(ctxPKE.sk,  $\mathcal{R}^2$ )
7: skPKE ← seedPKE.sk

// Compute public key pk
8: ctxPKE.pk ← XOF.Init(seedPKE.pk)
9: (ctxPKE.pk, h) ← SampleVect(ctxPKE.pk,  $\mathcal{R}$ )
10: s ← x + h · y
11: G'' ← ExpandMat§(G,  $n' - n$ )
12: G' ← TG''P
13: pkPKE ← (seedPKE.pk, s, G')
14: return (pkPKE, skPKE)

```

Algorithm 2 HEP-QC-PKE.Encrypt(pk_{PKE}, **m**, θ)

Require: Public key pk, message sk, randomness θ .

Ensure: Ciphertext **c**.

```

// Parse public key pk
1: seedPKE.pk ← pkPKE[0 : |seed|]
2: ctxPKE.pk ← XOF.Init(seedPKE.pk)
3: (ctxPKE.pk, h) ← SampleVect(ctxPKE.pk,  $\mathcal{R}$ )
4: s ← pkPKE[|seed| : |seed| + |s|]
5: G' ← pkPKE[|seed| + |s| : |seed| + |s| + |G'|]

// Compute ciphertext c
6: ctxθ ← XOF.Init( $\theta$ )
7: (ctxθ, r2) ← SampleFixedWeightVect(ctxθ,  $\mathcal{R}_{\omega_r}$ )
8: (ctxθ, e) ← SampleFixedWeightVect(ctxθ,  $\mathcal{R}_{\omega_e}$ )
9: (ctxθ, r1) ← SampleFixedWeightVect(ctxθ,  $\mathcal{R}_{\omega_r}$ )
10: u ← r1 + h · r2
11: v ← mG' + ExpandVect(s · r2 + e,  $n' - n$ )
12: c ← (u, v)
13: return c

```

Algorithm 3 HEP-QC-PKE.Decrypt(sk, c)

Require: Secret key sk, ciphertext c.**Ensure:** Message m.

```
// Parse secret key sk
1: seedPKE.sk ← skPKE[0 : |seed|]
2: ctxPKE.sk ← XOF.Init(seedPKE.sk)
3: (ctxPKE.sk, y) ← SampleFixedWeightVect§(ctxPKE.sk, Rω)
4: (ctxPKE.sk, P) ← GeneratePerm§(ctxPKE.sk, R)
5: (ctxPKE.sk, T) ← GenerateMat§(ctxPKE.sk, R2)

// Parse ciphertext c
6: u ← c[0 : |u|]
7: v ← c[|u| : |u| + |v|]

// Compute message m
8: c' ← v − ExpandVect(u · y, n' − n)
9: m' ← C.Decode(TruncateVect(c'P−1, n' − n1n2))
10: m ← m'T−1
11: return m
```

3.7 KEM

KEM is derived from PKE using the Fujisaki-Okamoto transformation [15] instantiating following the HHK framework with implicit rejection [24].

Algorithm 4 HEP-QC-KEM.KeyGen(seed_{PKE})

Require: None.**Ensure:** Encapsulation key ek_{KEM}, decapsulation key dk_{KEM}.

```
// Sample seedKEM
1: (seedKEM) ←§ B|seed|

// Compute seedPKE and randomness σ
2: ctxKEM ← XOF.Init(seedKEM)
3: (ctxKEM, seedPKE ← XOF.GetBytes(ctxKEM, |seed|)
4: (ctxKEM, σ) ← XOF.GetBytes(ctxKEM, |k|)

// Compute PKE keypair
5: (pkPKE, skPKE) ← HEP-QC-PKE.Keygen(seedPKE)

// Compute KEM keypair
6: ekKEM ← pkPKE
7: dkKEM ← (ekKEM, skPKE, σ, seedKEM)
8: return (ekKEM, dkKEM)
```

Algorithm 5 HEP-QC-KEM.Encaps(ek_{KEM})

Require: Encapsulation key ek_{KEM}

Ensure: Shared secret key K , ciphertext c_{KEM}

// Sample message m and salt

1: $m \leftarrow_{\$} \mathcal{B}^{|k|}$

2: $\text{salt} \leftarrow_{\$} \mathcal{B}^{|\text{salt}|}$

// Compute shared key K and ciphertext c_{KEM}

3: $(K, \theta) \leftarrow G(H(\text{ek}_{\text{KEM}}) \| m \| \text{salt})$

4: $c_{\text{PKE}} \leftarrow \text{HEP-QC-PKE.Encrypt}(\text{ek}_{\text{KEM}}, m, \theta)$

5: $c_{\text{KEM}} \leftarrow (c_{\text{PKE}}, \text{salt})$

6: **return** (K, c_{KEM})

Algorithm 6 HEP-QC-KEM.Decaps($\text{dk}_{\text{KEM}}, c_{\text{KEM}}$)

Require: Decapsulation key dk_{KEM} , ciphertext c_{KEM}

Ensure: Shared secret key K'

// Parse decapsulation key dk_{KEM}

1: $\text{ek}_{\text{KEM}} \leftarrow \text{dk}_{\text{KEM}}[0 : |\text{ek}_{\text{KEM}}|]$

2: $\text{sk}_{\text{PKE}} \leftarrow \text{dk}_{\text{KEM}}[|\text{ek}_{\text{KEM}}| : |\text{ek}_{\text{KEM}}| + |\text{sk}_{\text{PKE}}|]$

3: $\sigma \leftarrow \text{dk}_{\text{KEM}}[|\text{ek}_{\text{KEM}}| + |\text{sk}_{\text{PKE}}| : |\text{ek}_{\text{KEM}}| + |\text{sk}_{\text{PKE}}| + |\sigma|]$

// Parse ciphertext c_{KEM}

4: $c_{\text{PKE}} \leftarrow c_{\text{KEM}}[0 : |c_{\text{PKE}}|]$

5: $\text{salt} \leftarrow c_{\text{KEM}}[|c_{\text{PKE}}| : |c_{\text{PKE}}| + |\text{salt}|]$

6: **// Compute message m'**

7: $m' \leftarrow \text{HEP-QC-PKE.Decrypt}(\text{sk}_{\text{PKE}}, c_{\text{PKE}})$

// Compute shared key K' and ciphertext c'_{KEM}

8: $(K', \theta') \leftarrow G(H(\text{ek}_{\text{KEM}}) \| m' \| \text{salt})$

9: $c'_{\text{PKE}} \leftarrow \text{HEP-QC-PKE.Encrypt}(\text{ek}_{\text{KEM}}, m', \theta')$

10: $c'_{\text{KEM}} \leftarrow (c'_{\text{PKE}}, \text{salt})$

// Compute rejection key $\bar{K}$

11: $\bar{K} \leftarrow J(H(\text{ek}_{\text{KEM}}) \| \sigma \| c_{\text{KEM}})$

12: **if** $m' = \perp$ **or** $c'_{\text{KEM}} \neq c_{\text{KEM}}$ **then**

13: $K' \leftarrow \bar{K}$

14: **end if**

15: **return** K'

4. Parameters and Sizes

4.1 Parameter sets

The parameter sets of HEP-QC are given in Table 4.1, where n_1 denotes the length of the Reed-Solomon code and n_2 denotes the length of the Reed-Muller code. The parameter n denotes the length of the ambient space namely the smallest primitive prime greater than $n_1 n_2$. The parameters ω , ω_r and ω_e denote the weight of the vectors $(\mathbf{x}, \mathbf{y})$, $(\mathbf{r}_1, \mathbf{r}_2)$ and $\mathbf{e}$ respectively.

Instance	Security	n_1	n_2	n	k	ω	$\omega_r = \omega_e$	DFR
HEP-QC-1	128	46	384	17669	128	66	75	$< 2^{-128}$
HEP-QC-3	192	56	640	35851	192	100	114	$< 2^{-192}$
HEP-QC-5	256	90	640	57637	256	131	149	$< 2^{-256}$
HEP-QC-7	512	220	896	197123	512	261	297	$< 2^{-512}$

Table 4.1: HEP-QC parameter sets and security levels

4.2 Ciphertext and key sizes

The encapsulation key ek_{KEM} has size $|\text{ek}_{\text{KEM}}| = |\text{seed}| + \lceil n/8 \rceil + 8k \cdot \lceil n/64 \rceil$ while the decapsulation key dk_{KEM} has size $|\text{dk}_{\text{KEM}}| = |\text{ek}_{\text{KEM}}| + |\text{seed}| + \lceil k/8 \rceil + |\text{seed}|$. The ciphertext c_{KEM} has size $|\text{c}_{\text{KEM}}| = \lceil n/8 \rceil + \lceil (n_1 n_2)/8 \rceil + |\text{salt}|$ while the shared key K has size $|K|$. The seeds, salt and shared key have size $|\text{seed}| = 32 \text{ B}$, $|\text{salt}| = 16 \text{ B}$, $|K| = 32 \text{ B}$ respectively. The resulting sizes are given in bytes in Table 4.2.

Instance	Security	$ \text{ek}_{\text{KEM}} $	$ \text{dk}_{\text{KEM}} $	$ \text{c}_{\text{KEM}} $	$ K $
HEP-QC-1	128	285889	285969	4433	32
HEP-QC-3	192	866210	866298	8978	32
HEP-QC-5	256	1852485	1852581	14421	32
HEP-QC-7	512	12644449	12644577	49297	32

Table 4.2: HEP-QC keypair $(\text{ek}_{\text{KEM}}, \text{dk}_{\text{KEM}})$, ciphertext c_{KEM} and shared key K sizes (in Bytes)

5. Performance Analysis

Denote by $M(n)$ the complexity of one polynomial multiplication in $\mathcal{R}$. In general, $M(n) = O(n^2)$, while it can be reduced to $M(n) = O(n \log n)$ by using fast multiplication algorithms such as FFT or NTT. Let $D(n)$ denote the complexity of the decoding algorithm of the underlying code $\mathcal{C}$.

For KeyGen, the main computational cost comes from the generation of the public key. In particular, the computation $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$ requires one polynomial multiplication, while $\mathbf{G}' = \mathbf{T}\mathbf{G}''\mathbf{P}$ requires matrix transformations, where $\mathbf{T}$ is an invertible matrix and $\mathbf{P}$ is a permutation matrix. Since the matrix operation dominates, the time complexity of key generation is $T_{\text{KeyGen}} = O(k^2n')$. For Encaps, the main cost comes from the PKE encryption procedure. The computation of $\mathbf{u} = \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2$ and $\mathbf{v} = \mathbf{m}\mathbf{G}' + \text{ExpandVect}(\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, n' - n)$ involves two polynomial multiplications and one matrix-vector multiplication. Hence, the time complexity is $T_{\text{Encaps}} = O(M(n') + kn')$. For Decaps, the dominant operations are decryption and re-encryption. In particular, the decryption step contains one polynomial multiplication $\mathbf{u} \cdot \mathbf{y}$, one decoding operation of the code $\mathcal{C}$, and one matrix multiplication by $\mathbf{T}^{-1}$. Therefore, $T_{\text{Decaps}} = O(M(n') + D(n) + kn')$. Since the decoding algorithm is usually the most expensive part, in practice we have $T_{\text{Decaps}} = O(D(n))$.

5.1 System Configuration

The benchmarks were performed on a machine with the following configuration:

- Processor: Intel Core i7-14600K (14 cores, 3.5 GHz, 5.3 GHz turbo)
- Memory: 32 GB DDR5-5600
- Operating system: Ubuntu 24.04.4 LTS

The scheme have been compiled with gcc (version 13.3.0)

5.2 Performance Test Results

We evaluate the performance of our scheme at three security levels (Level 1, 3, and 5). For each level, we report the median cycle counts obtained using the `rdtsc` instruction. The results are summarized in Table 5.2.

Instance	KeyGen	Encaps	Decaps
HEP-QC-1	152064	10448	152592
HEP-QC-3	343323	30803	321945
HEP-QC-5	575858	71847	541570
HEP-QC-7	3399136	486252	2146718

Table 5.1: Performances (in kiloCycles) of the reference implementation of HEP-QC

5.3 Known Answer Test values

Known Answer Test values have been generated using the script provided by the Institute of Commercial Cryptography Standards. They are available in the folders `Test_Vectors/Reference_Implementation/hep-qc-x/` and `Test_Vectors/Optimized_Implementation/hep-qc-x/`. In addition, examples with intermediate values have also been provided in these folders.

6. Security

6.1 Choices of parameters

This section provides parameters sets for HEP-QC. Let n_1 and n_2 denote the length of the external Reed-Solomon code and the internal Reed-Muller code, respectively. Let n be the smallest primitive prime greater than $n_1 n_2$, $n' = 64 \lceil n/64 \rceil$ and $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$ denote the quotient ring. We introduce the private key at first.

- 1) The private key $(\mathbf{x}, \mathbf{y}) \in \mathcal{R}^2$ is a randomly generated such that $\text{wt}(\mathbf{x}) = \text{wt}(\mathbf{y}) = \omega$. To keep low decoding failure rate, ω is a positive integer in the range of $\frac{\sqrt{n}}{2}$.
- 2) We randomly generate a non-singular matrix $\mathbf{T} \in \mathbb{F}_2^{k \times k}$ and a permutation matrix $\mathbf{P} \in \mathbb{F}_2^{n' \times n'}$. It is used to compute the matrix $\mathbf{G}'$ and get the message.

The parameters of public key are listed as follows.

- 1) We randomly generate an element $\mathbf{e} \in \mathcal{R}$ such that $\text{wt}(\mathbf{e}) = \omega_e$ and a pair $(\mathbf{r}_1, \mathbf{r}_2) \in \mathcal{R}^2$ such that $\text{wt}(\mathbf{r}_1) = \text{wt}(\mathbf{r}_2) = \omega_r$. To keep low decoding failure rate, let ω_e, ω_r be positive integers all in the range of $\sqrt{n}/2$.
- 2) To maintain high decoding efficiency and low decoding failure rate, we also use the concatenated code to encode and decode in our scheme. The generator matrix of the concatenated code is $\mathbf{G} \in \mathbb{F}_2^{k \times n}$.
- 3) In order to improve the security of the scheme, the structure of $\mathbf{G}$ is disguised. A matrix $\mathbf{G}' \in \mathbb{F}_2^{k \times n'}$ is obtained by complex transformations of $\mathbf{G}$, then $\mathbf{G}'$ appears to be a random matrix, making decoding the linear code generated by $\mathbf{G}'$ a difficult problem.

6.2 Decoding Failure Rate analysis

The resulting decoding failure rate of the scheme is studied in this section. We analyze the distribution of the error vector $\mathbf{e}' = \text{TruncateVect}(\text{ExpandVect}(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}, n' - n)\mathbf{P}, n' - n)$, and then give the decoding failure rate of the concatenated code.

6.2.1 Analysis of the error vector distribution

Denote

$$\mathbf{e}' = (e'_0, e'_1, \dots, e'_{n'-1})$$

and

$$\mathbf{e}'' := \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e} = (e''_0, e''_1, \dots, e''_{n-1}).$$

We want the distribution of the Hamming weight $\text{wt}(\mathbf{e}')$, note that $\mathbf{e}'$ is obtained by applying a uniformly random $n' \times n'$ permutation matrix $\mathbf{P}$ to the vector $(\mathbf{e}'', \mathbf{0}_{n'-n}) \in \mathbb{F}_2^{n'}$ and then deleting the the last $n' - n$ coordinates. In the following, we analyze the distribution of the vector $\mathbf{e}''$ at first.

The vectors $\mathbf{x}, \mathbf{y}, \mathbf{r}_1, \mathbf{r}_2, \mathbf{e}$ have been taken uniformly random and independently chosen among vectors of weight ω, ω_r and ω_e . The distributions of the products $\mathbf{x} \cdot \mathbf{r}_2$ and $\mathbf{r}_1 \cdot \mathbf{y}$ have been evaluated in [1]:

Proposition 6.2.1. *Let $\mathbf{x} = (x_0, \dots, x_{n-1})$ be a random vector chosen uniformly among all binary vectors of weight ω and let $\mathbf{r} = (r_0, \dots, r_{n-1})$ be a random vector chosen uniformly among all vectors of weight*

ω_r and independently of $\mathbf{x}$. Then, denoting $\mathbf{z} = \mathbf{x} \cdot \mathbf{r}$, we have that for every $k \in \{0, \dots, n-1\}$, the k -th coordinate z_k of $\mathbf{z}$ is Bernoulli distributed with parameter $\tilde{p} = P(z_k = 1)$ equal to:

$$\tilde{p} = \frac{1}{\binom{n}{\omega} \binom{n}{\omega_r}} \sum_{\substack{1 \leq \ell \leq \min(\omega, \omega_r) \\ \ell \text{ odd}}} C_\ell,$$

where $C_\ell = \binom{n}{\ell} \binom{n-\ell}{\omega-\ell} \binom{n-\omega}{\omega_r-\ell}$.

By Proposition 6.2.1, the k -th coordinates of $\mathbf{x} \cdot \mathbf{r}_2$ and of $\mathbf{r}_1 \cdot \mathbf{y}$ are independent, and they are Bernoulli distributed with parameter $\tilde{p}$. It is then deduced that their modulo 2 sum $\mathbf{t} = \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y}$ is Bernoulli distributed with:

$$\begin{cases} \Pr[t_k = 1] = 2\tilde{p}(1 - \tilde{p}), \\ \Pr[t_k = 0] = (1 - \tilde{p})^2 + \tilde{p}^2. \end{cases}$$

Therefore the distribution of the coordinates of the vector $\mathbf{e}''$ are obtained:

Proposition 6.2.2. *Let $\mathbf{x}, \mathbf{y}, \mathbf{r}_1, \mathbf{r}_2, \mathbf{e}$ be independent random vectors with uniform distributions among vectors of fixed weight ω for $\mathbf{x}, \mathbf{y}$, among vectors of weight ω_r for $\mathbf{r}_1, \mathbf{r}_2$, and among vectors of weight ω_e for $\mathbf{e}$. Let $\mathbf{e}'' = \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e} = (e''_0, \dots, e''_{n-1})$. Then for any $k = 0 \dots n-1$, the coordinate e''_k has distribution:*

$$\begin{cases} \Pr[e''_k = 1] = 2\tilde{p}(1 - \tilde{p})(1 - \frac{\omega_e}{n}) + ((1 - \tilde{p})^2 + \tilde{p}^2) \frac{\omega_e}{n}, \\ \Pr[e''_k = 0] = ((1 - \tilde{p})^2 + \tilde{p}^2) (1 - \frac{\omega_e}{n}) + 2\tilde{p}(1 - \tilde{p}) \frac{\omega_e}{n}. \end{cases}$$

The above proposition gives the probability that a coordinate of the vector $\mathbf{e}''$ is 1. We assume that the coordinates of $\mathbf{e}''$ are independent, meaning that the weight of $\mathbf{e}''$ follows a binomial distribution of parameter p^* , where p^* is defined as in above proposition, that is

$$p^* = 2\tilde{p}(1 - \tilde{p})(1 - \frac{\omega_e}{n}) + ((1 - \tilde{p})^2 + \tilde{p}^2) \frac{\omega_e}{n}. \quad (6.1)$$

We are ready to determine the distribution of the error vector $\mathbf{e}'$.

Proposition 6.2.3. *Let $\mathbf{x}, \mathbf{y}, \mathbf{r}_1, \mathbf{r}_2, \mathbf{e}$ be independent random vectors with uniform distributions among vectors of fixed weight ω for $\mathbf{x}, \mathbf{y}$, among vectors of weight ω_r for $\mathbf{r}_1, \mathbf{r}_2$, and among vectors of weight ω_e for $\mathbf{e}$. Let $\mathbf{P}$ be independent random matrix with uniform distributions among permutation matrix and $\mathbf{e}' = \text{TruncateVect}(\text{ExpandVect}(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}, n' - n)\mathbf{P}, n' - n) = \text{TruncateVect}((\mathbf{e}'', \mathbf{0}_{n'-n})\mathbf{P}, n' - n) = (e'_0, \dots, e'_{n'-1})$. Then*

$$\Pr(\text{wt}(\mathbf{e}') = d) = \sum_{k=\max(0, 2n-n')}^n \frac{\binom{n}{k} \binom{n'-n}{n-k}}{\binom{n'}{n}} \binom{k}{d} p^{*d} (1 - p^*)^{k-d},$$

where we interpret $\binom{k}{d} = 0$ when $d > k$ and p^* is defined in Equation (6.1).

Proof. Since $\mathbf{P}$ is uniformly random, the error vector $(e'_1, \dots, e'_{n'})$ is exactly a simple random sample of size n drawn without replacement from the n' coordinates of $(\mathbf{e}'', \mathbf{0}_{n'-n})$. Among these n' coordinates, the first n are $\{e''_0, \dots, e''_{n-1}\}$ (each equals 1 with probability p^*), and the remaining $n' - n$ are deterministic zeros. Define the random variable

$$K = \text{number of the } n \text{ sampled coordinates that come from } \{e''_0, \dots, e''_{n-1}\}.$$

Since we draw n items without replacement from a set of size n' that contains exactly n special items (the positions of the e_i''), K follows the distribution:

$$\Pr(K = k) = \frac{\binom{n}{k} \binom{n'-n}{n-k}}{\binom{n'}{n}}, \quad k = \max(0, 2n - n'), \dots, n.$$

Given $K = k$, the sample consists of k i.i.d. Bernoulli(p^*) variables and $n - k$ zeros. Hence, conditionally on $K = k$, the weight $\mathbf{e}'$ is the sum of these k Bernoulli trials:

$$\Pr(\text{wt}(\mathbf{e}') = d \mid K = k) = \binom{k}{d} p^{*d} (1 - p^*)^{k-d}, \quad d = 0, 1, \dots, k,$$

and the probability is 0 for $d > k$. Now use the law of total probability:

$$\Pr(\text{wt}(\mathbf{e}') = d) = \sum_k \Pr(K = k) \Pr(\text{wt}(\mathbf{e}') = d \mid K = k).$$

Inserting the expressions above gives

$$\Pr(\text{wt}(\mathbf{e}') = d) = \sum_{k=\max(0, 2n-n')}^n \frac{\binom{n}{k} \binom{n'-n}{n-k}}{\binom{n'}{n}} \binom{k}{d} p^{*d} (1 - p^*)^{k-d}.$$

This completes the proof. $\square$

6.2.2 Decoding failure rate analysis

A lower bound on the decoding probability of duplicated Reed-Muller code with dimension 8 used in HEP-QC has been derived in [1].

Proposition 6.2.4. *Let p be the transition probability of the binary symmetric channel. Then the DFR of a duplicated Reed-Muller code of dimension 8 and minimal distance d_i can be upper bounded by:*

$$p_i = 255 \sum_{j=d_i/2}^{d_i} \binom{d_i}{j} p^j (1 - p)^{d_i-j}.$$

Using the lower bound p_i on the decoding probability of the Reed-Muller codes given in proposition 6.2.4, one can deduce the DFR of the concatenated code.

Proposition 6.2.5. *The DFR of the concatenated code using a Reed-Solomon code $[n_e, k_e, d_e]_{\mathbb{F}_{256}}$ as the external code and a Reed-Muller code as the internal code can be upper bounded by:*

$$\sum_{l=\delta_e+1}^{n_e} \binom{n_e}{l} p_i^l (1 - p_i)^{n_e-l}.$$

6.3 Security Statement and Analysis

We now describe the security proof strategy for HEP-QC. First, we establish the IND-CPA security of HEP-QC-PKE in Section 6.3.1. Then, in Section 6.3.2, we prove the IND-CCA2 security of HEP-QC-KEM obtained via the salted Fujisaki–Okamoto transformation with implicit rejection (SFO[†]) [15, 20, 24]. For ease of presentation, throughout Sections 6.3.1 and 6.3.2 we assume that the vectors $\mathbf{r}_1, \mathbf{r}_2$ and $\mathbf{e}$ are sampled using `SampleFixedWeightVects` rather than `SampleFixedWeightVect` (see Section 3.2). Finally,

in Section 6.3.3, we show that replacing `SampleFixedWeightVects` by `SampleFixedWeightVect` only has a negligible effect on the validity of the preceding security proofs.

6.3.1 IND-CPA security

We will prove the IND-CPA security of the HEP-QC-PKE in this section.

Theorem 6.3.1. *For any IND-CPA adversary $\mathcal{A}$ against the HEP-QC-PKE scheme, there exist adversaries $\mathcal{B}_1$, $\mathcal{B}_2$, and $\mathcal{B}_3$ such that*

$$\text{Adv}_{\text{HEP-QC}}^{\text{IND-CPA}}(\mathcal{A}) \leq \min \left\{ \text{Adv}_{\text{EPC-P}}(\mathcal{B}_1), \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_2) + \text{Adv}_{3\text{-DQCSD-P}}(\mathcal{B}_3) \right\}.$$

Proof. We prove the theorem by deriving two independent upper bounds on the IND-CPA advantage of any adversary $\mathcal{A}$. The first bound is obtained by a reduction to the EPC-P problem, while the second bound is obtained by a joint reduction to the 2-DQCSD-P and 3-DQCSD-P problems. In both arguments, the real PKE.IND-CPA experiment is gradually transformed into an experiment in which the challenge ciphertext is independent of the challenge bit. Taking the minimum of the two resulting bounds gives the claimed inequality. For simplicity, the distinguishers are given the matrix $\mathbf{H}$ directly, rather than $\text{seed}_{\text{PKE}, \text{pk}}$ used to generate $\mathbf{h}$ and then $\mathbf{H}$.

First bound: reduction to the EPC-P problem. We first prove the first bound by a sequence-of-games argument. Starting from the real PKE.IND-CPA experiment, Game_2 is obtained from Game_1 by replacing the structured distribution associated with the public-key component $\mathbf{G}'$ by the corresponding uniform distribution. This transition is reduced to the EPC-P problem. The analysis of Game_2 then yields the desired IND-CPA bound.

Game₁. This is the real PKE.IND-CPA game.

Game ₁ (λ)
1. $\text{seed}_{\text{PKE}} \xleftarrow{\\$} \mathcal{B}^{ \text{seed} }$ 2. $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{HEP-QC.KeyGen}(\text{seed}_{\text{PKE}})$ 3. $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{\text{CHOOSE}}(\text{pk}_{\text{PKE}})$ 4. $b \xleftarrow{\\$} \{0, 1\}$ 5. $\theta \xleftarrow{\\$} \mathcal{B}^{ \theta }$, $\text{c}_{\text{PKE}} \leftarrow \text{HEP-QC.Encrypt}(\text{pk}_{\text{PKE}}, \mathbf{m}_b, \theta)$ 6. $b' \leftarrow \mathcal{A}_{\text{GUESS}}(\text{pk}_{\text{PKE}}, \text{c}_{\text{PKE}})$ 7. return $(b = b')$

Game₂. In this game, the simulator does not generate the decryption key and instead samples $\mathbf{G}'$ at random in $\mathbb{F}_2^{k \times n}$. The remaining steps are unchanged from Game_1 .

Game ₂ (λ)
1. $\text{seed}_{\text{PKE}} \xleftarrow{\\$} \mathcal{B}^{ \text{seed} }$ 2. 1. $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{HEP-QC.Keygen}(\text{seed}_{\text{PKE}})$ 2. $\mathbf{G}' \xleftarrow{\\$} \mathbb{F}_2^{k \times n'}$ 3. $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow ((\text{seed}_{\text{PKE}, \text{pk}}, \mathbf{s}, \mathbf{G}'), 0)$ 3. $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{\text{CHOOSE}}(\text{pk}_{\text{PKE}})$ 4. $b \xleftarrow{\\$} \{0, 1\}$ 5. $\theta \xleftarrow{\\$} \mathcal{B}^{ \theta }$, $\text{c}_{\text{PKE}} \leftarrow \text{HEP-QC.Encrypt}(\text{pk}_{\text{PKE}}, \mathbf{m}_b, \theta)$ 6. $b' \leftarrow \mathcal{A}_{\text{GUESS}}(\text{pk}_{\text{PKE}}, \text{c}_{\text{PKE}})$ 7. return $(b = b')$

The only difference between Game_1 and Game_2 is the distribution of $\mathbf{G}'$ in the encryption key $\text{pk}_{\text{PKE}} = (\mathbf{H}, \mathbf{s}, \mathbf{G}')$. In Game_1 , conditioned on $\mathbf{H}$ and $\mathbf{s}$, Game_1 samples $\mathbf{G}'$ according to the structured distribution induced by the $\text{EPC-P}(k, n', n_1 n_2)$ instance, while Game_2 replaces it with a uniform matrix over $\mathbb{F}_2^{k \times n'}$. Thus, distinguishing these two games is reduced to distinguishing the structured distribution of $\mathbf{G}'$ from the corresponding uniform distribution.

Suppose that an adversary $\mathcal{B}_1$ is able to distinguish Game_1 from Game_2 with advantage ϵ for some security parameter λ . Then one can build an algorithm $\mathcal{D}_{\text{EPC-P}}^\lambda$ solving the EPC-P problem with the same advantage ϵ .

$\mathcal{D}_{\text{EPC-P}}^\lambda(\mathbf{H}, \mathbf{s}, \mathbf{G}')$

1. Compute $\mathbf{h}$ from $\mathbf{H} = (\mathbf{I}_n \text{ rot}(\mathbf{h}))$
2. Compute $\text{pk}_{\text{PKE}} \leftarrow (\mathbf{h}, \mathbf{s}, \mathbf{G}')$
3. Get $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{B}_{1, \text{CHOOSE}}(\text{pk}_{\text{PKE}})$
4. Sample $b \xleftarrow{\$} \{0, 1\}$ and $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$
5. Compute $\mathbf{c}_{\text{PKE}} \leftarrow \text{HEP-QC.Encrypt}(\text{pk}_{\text{PKE}}, \mathbf{m}_b, \theta)$
6. Get $b' \leftarrow \mathcal{B}_{1, \text{GUESS}}(\text{pk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$
7. If $b' = \text{Game}_1$, output the $\text{EPC-P}(k, n', n_1 n_2)$ distribution
8. If $b' = \text{Game}_2$, output the $\mathcal{U}(\mathbb{F}_2^{k \times n'} \times \mathbb{F}_2^{k \times n_1 n_2})$ distribution

Hence,

$$|\Pr[\text{Game}_1 \Rightarrow 1] - \Pr[\text{Game}_2 \Rightarrow 1]| \leq \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_1).$$

Since the distribution of $\mathbf{c}_{\text{PKE}}$ is independent of the challenge bit b in Game_2 . Then we have

$$\Pr[\text{Game}_2 \Rightarrow 1] = \frac{1}{2}.$$

The advantage of $\mathcal{A}$ in the real IND-CPA game satisfies

$$\begin{aligned} \text{Adv}_{\text{HEP-QC}}^{\text{IND-CPA}}(\mathcal{A}) &= \left| \Pr[\text{Game}_1 \Rightarrow 1] - \frac{1}{2} \right| \\ &= |\Pr[\text{Game}_1 \Rightarrow 1] - \Pr[\text{Game}_2 \Rightarrow 2]| \\ &\leq |\Pr[\text{Game}_1 \Rightarrow 1] - \Pr[\text{Game}_2 \Rightarrow 2]| \\ &\leq \text{Adv}_{\text{EPC-P}}(\mathcal{B}_1). \end{aligned}$$

Second bound: joint reduction to the 2-DQCSD-P and 3-DQCSD-P problems. We now prove the second bound by a sequence-of-games argument. Starting from the real PKE.IND-CPA experiment, we gradually modify the distributions involved in the public key and in the challenge ciphertext. The transition from Game_1 to Game_2 replaces the structured public-key distribution by the corresponding uniform distribution with prescribed parity, and this transition is reduced to the 2-DQCSD-P problem. The transition from Game_2 to Game_3 replaces the ciphertext-related distribution by the corresponding uniform distribution with prescribed parity, and this transition is reduced to the 3-DQCSD-P problem. In the final game, the challenge ciphertext is independent of the challenge bit, which gives the desired IND-CPA bound. Let

$$b_1 = h(1) \bmod 2, \quad b_2 = \omega + b_1 \times \omega \bmod 2,$$

$$b_3 = \omega_r + b_1 \times \omega_r \bmod 2 = \omega_e + b_1 \times \omega_e \bmod 2.$$

Game₁. This is the real PKE.IND-CPA game.

Game₁(λ)

1. $\text{seed}_{\text{PKE}} \xleftarrow{\$} \mathcal{B}^{|\text{seed}|}$
2. $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{HEP-QC.Keygen}(\text{seed}_{\text{PKE}})$
3. $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{\text{CHOOSE}}(\text{pk}_{\text{PKE}})$
4. $b \xleftarrow{\$} \{0, 1\}$
5. $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$, $\mathbf{c}_{\text{PKE}} \leftarrow \text{HEP-QC.Encrypt}(\text{pk}_{\text{PKE}}, \mathbf{m}_b, \theta)$
6. $b' \leftarrow \mathcal{A}_{\text{GUESS}}(\text{pk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$
7. **return** $(b = b')$

Game₂. In this game, the decryption key is discarded, and the vector $\mathbf{s}$ is sampled uniformly with parity b_2 . The remaining steps are the same as in Game₁.

Game₂(λ)

1. $\text{seed}_{\text{PKE}} \xleftarrow{\$} \mathcal{B}^{|\text{seed}|}$
2.

1. $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{HEP-QC.Keygen}(\text{seed}_{\text{PKE}})$
 2. $\mathbf{s} \xleftarrow{\$} \mathbb{F}_{2, b_2}^n$
 3. $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow ((\text{seed}_{\text{PKE.pk}}, \mathbf{s}, \mathbf{G}'), 0)$
3. $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{\text{CHOOSE}}(\text{pk}_{\text{PKE}})$
4. $b \xleftarrow{\$} \{0, 1\}$
5. $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$, $\mathbf{c}_{\text{PKE}} \leftarrow \text{HEP-QC.Encrypt}(\text{pk}_{\text{PKE}}, \mathbf{m}_b, \theta)$
6. $b' \leftarrow \mathcal{A}_{\text{GUESS}}(\text{pk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$
7. **return** $(b = b')$

The only difference between Game₁ and Game₂ is the distribution of $\mathbf{s}$ in the encryption key $\text{pk}_{\text{PKE}} = (\mathbf{H}, \mathbf{s}, \mathbf{G}')$. In Game₁, conditioned on $\mathbf{H}$ and $\mathbf{G}'$, the vector $\mathbf{s}$ is generated according to the structured distribution induced by the 2-QCSD-P(n, ω, b_1, b_2) instance, whereas in Game₂ it is sampled uniformly from $\mathbb{F}_{2, b_2}^n$. Hence, distinguishing these two games is reduced to distinguishing the structured distribution of $\mathbf{s}$ from the corresponding uniform distribution.

Therefore, if there exists a distinguisher $\mathcal{B}_2$ that distinguishes Game₁ from Game₂ with advantage ϵ , then one can construct an algorithm $\mathcal{D}_{2\text{-DQCSD-P}}^\lambda$ that solves the 2-DQCSD-P problem with the same advantage ϵ .

$\mathcal{D}_{2\text{-DQCSD-P}}^\lambda(\mathbf{H}, \mathbf{s}, \mathbf{G}')$

1. Compute h from $\mathbf{H} = (\mathbf{I}_n \text{ rot}(\mathbf{h}))$
2. Compute $\text{pk}_{\text{PKE}} \leftarrow (\mathbf{h}, \mathbf{s}, \mathbf{G}')$
3. Get $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{B}_{2, \text{CHOOSE}}(\text{pk}_{\text{PKE}})$
4. Sample $b \xleftarrow{\$} \{0, 1\}$ and $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$
5. Compute $\mathbf{c}_{\text{PKE}} \leftarrow \text{HEP-QC.Encrypt}(\text{pk}_{\text{PKE}}, \mathbf{m}_b, \theta)$
6. Get $b' \leftarrow \mathcal{B}_{2, \text{GUESS}}(\text{pk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$
7. If $b' = \text{Game}_1$, output the 2-QCSD-P(n, ω, b_1, b_2) distribution
8. If $b' = \text{Game}_2$, output the $\mathcal{U}(\mathbb{F}_{2, b_1}^{n \times 2n} \times \mathbb{F}_{2, b_2}^n)$ distribution

Hence,

$$|\Pr[\text{Game}_1 \Rightarrow 1] - \Pr[\text{Game}_2 \Rightarrow 1]| \leq \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_2).$$

Game₃. In this game, instead of sampling correctly weighted vectors $\mathbf{r}_1, \mathbf{r}_2, \mathbf{e}$, the simulator samples uniformly random vectors from $\mathbb{F}_2^n$ and generates the challenge ciphertext subject to the prescribed parity condition.

Game₃(λ)

1. $\text{seed}_{\text{PKE}} \xleftarrow{\$} \mathcal{B}^{|\text{seed}|}$
2.

1. $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{HEP-QC.Keygen}(\text{seed}_{\text{PKE}})$
 2. $\mathbf{s} \xleftarrow{\$} \mathbb{F}_{2, b_2}^n$
 3. $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow ((\text{seed}_{\text{PKE.pk}}, \mathbf{s}, \mathbf{G}'), 0)$

- 3. $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{\text{CHOOSE}}(\text{pk}_{\text{PKE}})$
- 4. $b \xleftarrow{\$} \{0, 1\}$
- 5.

1. $\mathbf{e} \xleftarrow{\$} \mathbb{F}_2^n, (\mathbf{r}_1, \mathbf{r}_2) \xleftarrow{\$} \mathbb{F}_{2, \omega_r}^n \times \mathbb{F}_{2, \omega_r}^n$
 2. $\mathbf{u} \leftarrow \mathbf{r}_1 + h\mathbf{r}_2$
 3. $\mathbf{v} \leftarrow \mathcal{C}.\text{Encode}(\mathbf{m}_b) + \text{ExpandVect}(\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, n' - n)$
 4. $\mathbf{c}_{\text{PKE}} \leftarrow (\mathbf{u}, \mathbf{v})$

- 6. $b' \leftarrow \mathcal{A}_{\text{GUESS}}(\text{pk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$
- 7. **return** $(b = b')$

The transition from Game₂ to Game₃ is controlled by the 3-DQCSD-P assumption. Suppose that there exists a distinguisher $\mathcal{B}_3$ that distinguishes Game₂ from Game₃ with advantage ϵ . Then one can construct an algorithm $\mathcal{D}_{3\text{-DQCSD-P}}^\lambda$ solving the 3-DQCSD-P problem with the same advantage ϵ .

$\mathcal{D}_{3\text{-DQCSD-P}}^\lambda(\mathbf{H}, \mathbf{G}', (\mathbf{u}, \mathbf{v}))$

1. Compute h and $\mathbf{s}$ from $\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}$
2. Compute $\text{pk}_{\text{PKE}} \leftarrow (\mathbf{h}, \mathbf{s}, \mathbf{G}')$
3. Get $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{B}_{3, \text{CHOOSE}}(\text{pk}_{\text{PKE}})$
4. Sample $b \xleftarrow{\$} \{0, 1\}$
5. Compute $\mathbf{c}_{\text{PKE}} \leftarrow (\mathbf{u}, \mathcal{C}.\text{Encode}(\mathbf{m}_b) + \mathbf{v})$
6. Get $b' \leftarrow \mathcal{B}_{3, \text{GUESS}}(\text{pk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$
7. If $b' = \text{Game}_2$, output the 3-QCSD-P(n, ω, b_1, b_2, b_3) distribution
8. If $b' = \text{Game}_3$, output the $\mathcal{U}(\mathbb{F}_{2, b_1, b_2}^{2n \times 3n} \times (\mathbb{F}_{2, b_3}^n \times \mathbb{F}_2^n))$ distribution

Then we have

$$(\mathbf{u}, \mathbf{v} - \mathcal{C}.\text{Encode}(\mathbf{m}_b))^\top = ((\mathbf{I}_n \ \mathbf{0} \ \text{rot}(\mathbf{h})) \cdot (\mathbf{r}_1 \ \mathbf{e} \ \mathbf{r}_2)^\top, \text{ExpandVect}((\mathbf{0} \ \mathbf{I}_n \ \text{rot}(\mathbf{s})) \cdot (\mathbf{r}_1 \ \mathbf{e} \ \mathbf{r}_2)^\top, n' - n)).$$

Thus, the only difference between Game₂ and Game₃ is the distribution of

$$\left(\begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}, (\mathbf{u}, \mathbf{v} - \mathcal{C}.\text{Encode}(\mathbf{m}_b)) \right).$$

In Game₂, this pair follows the 3-QCSD-P(n, ω, b_1, b_2, b_3) distribution, whereas in Game₃ it follows the uniform distribution with the prescribed parity over $\mathbb{F}_{2, b_1, b_2}^{2n \times 3n} \times (\mathbb{F}_{2, b_3}^n \times \mathbb{F}_2^n)$. Therefore,

$$|\Pr[\text{Game}_2 \Rightarrow 1] - \Pr[\text{Game}_3 \Rightarrow 1]| \leq \text{Adv}_{3\text{-DQCSD-P}}(\mathcal{B}_3).$$

Finally, in Game₃, the distribution of $\mathbf{c}_{\text{PKE}}$ is independent of the challenge bit b . Indeed, $\mathbf{u}$ is generated independently of b , and $\mathbf{v}$ is masked by the random vector $\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}$. Hence,

$$\Pr[\text{Game}_3 \Rightarrow 1] = \frac{1}{2}.$$

By the triangle inequality, the advantage of $\mathcal{A}$ in the real IND-CPA game satisfies

$$\begin{aligned}
\text{Adv}_{\text{HEP-QC}}^{\text{IND-CPA}}(\mathcal{A}) &= \left| \Pr[\text{Game}_1 \Rightarrow 1] - \frac{1}{2} \right| \\
&= |\Pr[\text{Game}_1 \Rightarrow 1] - \Pr[\text{Game}_3 \Rightarrow 1]| \\
&\leq |\Pr[\text{Game}_1 \Rightarrow 1] - \Pr[\text{Game}_2 \Rightarrow 1]| + |\Pr[\text{Game}_2 \Rightarrow 1] - \Pr[\text{Game}_3 \Rightarrow 1]| \\
&\leq \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_2) + \text{Adv}_{3\text{-DQCSD-P}}(\mathcal{B}_3).
\end{aligned}$$

Combining the two bounds, namely

$$\text{Adv}_{\text{HEP-QC}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{\text{EPC-P}}(\mathcal{B}_1)$$

and

$$\text{Adv}_{\text{HEP-QC}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_2) + \text{Adv}_{3\text{-DQCSD-P}}(\mathcal{B}_3),$$

we obtain

$$\text{Adv}_{\text{HEP-QC}}^{\text{IND-CPA}}(\mathcal{A}) \leq \min \{ \text{Adv}_{\text{EPC-P}}(\mathcal{B}_1), \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_2) + \text{Adv}_{3\text{-DQCSD-P}}(\mathcal{B}_3) \}.$$

This completes the proof. $\square$

6.3.2 IND-CCA2 security

HEP-QC-KEM is obtained by applying the salted Fujisaki–Okamoto (SFO[†]) [15, 20, 24] transform to HEP-QC-PKE. In this section, we discuss the IND-CCA2 security of HEP-QC-KEM.

Definition 6.3.2 (δ -correct PKE [24]). A PKE = (PKE.KeyGen, PKE.Encrypt, PKE.Decrypt) is δ -correct if:

$$\mathbb{E} \left(\max_{\mathbf{m} \in \mathcal{M}} \Pr [\text{PKE.Decrypt}(\text{sk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}}) \neq \mathbf{m} \mid \mathbf{c}_{\text{PKE}} \leftarrow \text{PKE.Encrypt}(\text{pk}_{\text{PKE}}, \mathbf{m})] \right) \leq \delta. \quad (6.2)$$

where the expectation is taken over $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{PKE.KeyGen}(\text{param})$.

Definition 6.3.3 (δ -correct KEM [24]). A KEM = (KEM.KeyGen, KEM.Encaps, KEM.Decaps) is δ -correct if:

$$\Pr \left[\text{KEM.Decaps}(\text{sk}_{\text{KEM}}, \mathbf{c}_{\text{KEM}}) \neq K \mid \begin{array}{l} (\text{pk}_{\text{KEM}}, \text{sk}_{\text{KEM}}) \leftarrow \text{KEM.KeyGen}(); \\ (K, \mathbf{c}_{\text{KEM}}) \leftarrow \text{KEM.Encaps}(\text{pk}_{\text{KEM}}) \end{array} \right] \leq \delta. \quad (6.3)$$

In HEP-QC-PKE the failure to decrypt a ciphertext $\mathbf{c}_{\text{PKE}}$ occurs if and only if

$$\omega(\text{TruncateVect}(\text{ExpandVect}(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}, n' - n)\mathbf{P}, n' - n)) > \Delta. \quad (6.4)$$

Note that the aforementioned equation does not depend on the message $\mathbf{m}$. Therefore, the probability in Equation 9 simplifies to

$$\Pr [\text{PKE.Decrypt}(\text{sk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}}) \neq \mathbf{m} \mid \mathbf{c}_{\text{PKE}} \leftarrow \text{PKE.Encrypt}(\text{pk}_{\text{PKE}}, \mathbf{m})] \leq \delta. \quad (6.5)$$

This probability is equivalent to the probability that is analyzed in Section 6.3, namely:

$$\Pr \left[\begin{array}{l} \omega \left(\text{TruncateVect}(\text{ExpandVect} \right. \\ \left. (\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}, n' - n) \mathbf{P}, \right. \\ \left. n' - n \right) > \Delta \end{array} \middle| \begin{array}{l} \text{ctx}_{\text{PKE.sk}} \leftarrow \text{XOF.Init}(\text{seed}_{\text{PKE.sk}}); \\ (\text{ctx}_{\text{PKE.sk}}, \mathbf{y}) \leftarrow \text{SampleFixedWeightVect}_{\S}(\text{ctx}_{\text{PKE.sk}}, \mathcal{R}_{\omega}); \\ (\text{ctx}_{\text{PKE.sk}}, \mathbf{x}) \leftarrow \text{SampleFixedWeightVect}_{\S}(\text{ctx}_{\text{PKE.sk}}, \mathcal{R}_{\omega}); \\ \text{ctx}_{\theta} \leftarrow \text{XOF.Init}(\theta); \\ (\text{ctx}_{\theta}, \mathbf{r}_2) \leftarrow \text{SampleFixedWeightVect}_{\S}(\text{ctx}_{\theta}, \mathcal{R}_{\omega_r}); \\ (\text{ctx}_{\theta}, \mathbf{e}) \leftarrow \text{SampleFixedWeightVect}_{\S}(\text{ctx}_{\theta}, \mathcal{R}_{\omega_e}); \\ (\text{ctx}_{\theta}, \mathbf{r}_1) \leftarrow \text{SampleFixedWeightVect}_{\S}(\text{ctx}_{\theta}, \mathcal{R}_{\omega_r}) \end{array} \right] \leq \delta. \quad (6.6)$$

The next theorem shows that HEP-QC-KEM achieves IND-CCA2 security in the random-oracle model, with its concrete advantage bound obtained by combining the estimates of Theorem 13 from [20] and Theorem 6.3.1.

Theorem 6.3.4. *If HEP-QC-PKE is δ -correct, for any IND-CCA2 adversary $\mathcal{A}$ against the HEP-QC-KEM scheme issuing at most q_{RO} queries to G and q_D queries to the HEP-QC-KEM.Decaps oracle, there exists adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that:*

$$\begin{aligned} \text{Adv}_{\text{HEP-QC-KEM}}^{\text{IND-CCA2}}(\mathcal{A}) &\leq \frac{1}{2^{|k|-2|\text{salt}|}} + \frac{3q_{\text{RO}}}{2^{|k|}} + (q_{\text{RO}} + q_D) \cdot \delta \\ &\quad + 2 \cdot \min(\text{Adv}_{\text{EPC-P}}(\mathcal{B}_1), \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_2) + \text{Adv}_{3\text{-DQCSD-P}}(\mathcal{B}_3)). \end{aligned} \quad (6.7)$$

6.3.3 Security proof with non uniform randomness sampling

In this section, we analyze the effect of using `SampleFixedWeightVect` instead of the ideal uniform sampler `SampleFixedWeightVect\S`. Following the method of [39], we show that this replacement only slightly changes the IND-CCA2 security bound of HEP-QC-KEM.

Proposition 6.3.5. *[39] When $x \leftarrow \text{randbits}(B, \text{prng})$ behaves as a random oracle which yields uniformly distributed integers $0 \leq x < 2^B$ for any integer $B > 0$, one has:*

$$\begin{aligned} \prod_{i=0}^{\omega-1} \left(1 - \frac{n_i}{2^B} \right) &= \tau_{\min}^{\omega} \leq \frac{\Pr[\mathbf{e} \mid \mathbf{e} \leftarrow \text{SampleFixedWeightVect}(\text{ctx}_{\mathbf{e}}, \mathcal{R}_{\omega})]}{\Pr[\mathbf{e} \mid \mathbf{e} \leftarrow \text{SampleFixedWeightVect}_{\S}(\text{ctx}_{\mathbf{e}}, \mathcal{R}_{\omega})]} \\ &\leq \tau_{\max}^{\omega} = \prod_{i=0}^{\omega-1} \left(1 + \frac{(n-i) - n_i}{2^B} \right). \end{aligned} \quad (6.8)$$

where $n_i = 2^B \bmod (n-i)$ for all i , $0 \leq i < \omega$.

For the HEP-QC-KEM parameter sets, the quantities $(\tau_{\min}^{\omega}, \tau_{\max}^{\omega})$ and $(\tau_{\min}^{\omega_r}, \tau_{\max}^{\omega_r})$ are very close to 1. The next lemma, corresponding to Equation 5 in [39], states that replacing the uniform sampling of a vector $\mathbf{e}$ of weight ω by Algorithm `SampleFixedWeightVect` can increase the advantage of any adversary by at most the multiplicative factor $\tau_{\max}^{\omega}$.

Lemma 6.3.6. *For any real-valued random variable $V : \mathcal{R}_{\omega} \rightarrow \mathbb{R}$, one has:*

$$\begin{aligned} \sum_{\mathbf{e} \in \mathcal{R}_{\omega}} \Pr[\mathbf{e} \mid \mathbf{e} \leftarrow \text{SampleFixedWeightVect}(\text{ctx}_{\mathbf{e}}, \mathcal{R}_{\omega})] V(\mathbf{e}) &\leq \\ \tau_{\max}^{\omega} \cdot \sum_{\mathbf{e} \in \mathcal{R}_{\omega}} \Pr[\mathbf{e} \mid \mathbf{e} \leftarrow \text{SampleFixedWeightVect}_{\S}(\text{ctx}_{\mathbf{e}}, \mathcal{R}_{\omega})] V(\mathbf{e}). \end{aligned}$$

Impact on the security proof. Following [39], we now indicate how Equation 13 should be modified when the randomness is sampled from the non-uniform distribution:

- The first two terms

$$\frac{1}{2^{|k|-2|\text{salt}|}} \quad \text{and} \quad \frac{3q_{\text{RO}}}{2^{|\epsilon|}}$$

are not affected, since they do not depend on the sampler output;

- The third term $(q_{RO} + q_D) \cdot \delta$ comes from the δ -correctness condition of the scheme. If $(\mathbf{x}, \mathbf{y})$ and $(\mathbf{r}_1, \mathbf{r}_2, \mathbf{e})$ are sampled using `SampleFixedWeightVect` instead of the uniform sampler, then δ has to satisfy:

$$\Pr \left[\begin{array}{l} \omega \left(\text{TruncateVect}(\text{ExpandVect} \right. \\ \left. (\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}, n' - n) \mathbf{P}, \right. \\ \left. n' - n \right) > \Delta \end{array} \middle| \begin{array}{l} \text{ctx}_{\text{XPKE.sk}} \leftarrow \text{XOF.Init}(\text{seed}_{\text{PKE.sk}}); \\ (\text{ctx}_{\text{PKE.sk}}, \mathbf{y}) \leftarrow \text{SampleFixedWeightVect}_{\mathbb{S}}(\text{ctx}_{\text{PKE.sk}}, \mathcal{R}_{\omega}); \\ (\text{ctx}_{\text{PKE.sk}}, \mathbf{x}) \leftarrow \text{SampleFixedWeightVect}_{\mathbb{S}}(\text{ctx}_{\text{PKE.sk}}, \mathcal{R}_{\omega}); \\ \text{ctx}_{\theta} \leftarrow \text{XOF.Init}(\theta); \\ (\text{ctx}_{\theta}, \mathbf{r}_2) \leftarrow \text{SampleFixedWeightVect}(\text{ctx}_{\theta}, \mathcal{R}_{\omega_r}); \\ (\text{ctx}_{\theta}, \mathbf{e}) \leftarrow \text{SampleFixedWeightVect}(\text{ctx}_{\theta}, \mathcal{R}_{\omega_e}); \\ (\text{ctx}_{\theta}, \mathbf{r}_1) \leftarrow \text{SampleFixedWeightVect}(\text{ctx}_{\theta}, \mathcal{R}_{\omega_r}). \end{array} \right] \leq \delta.$$

By Lemma 6.3.6, this probability can grow by no more than the factor $(\tau_{\max}^{\omega_r})^3$;

- The fourth term

$$2 \cdot \min(\text{Adv}_{\text{EPC-P}}(\mathcal{B}_1), \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_2) + \text{Adv}_{3\text{-DQCSD-P}}(\mathcal{B}_3))$$

also needs to be updated to reflect the slight modification of the game sequence. One may insert an initial game Game_0 in which the small Hamming weight vectors are generated according to the biased sampler rather than the uniform sampler, before continuing with Game_1 . Lemma 6.4 then implies that the advantage corresponding to this term is multiplied by at most $(\tau_{\max}^{\omega_r})^3$.

6.3.4 Known attacks

According to Theorem 6.3.4, the attacker must attack two problems together. Consequently, we will analyze the complexity of classical attack and quantum attack on QCSD-P problem and EPC-P problem separately.

Attacks against Syndrome Decoding. The concrete hardness of the syndrome decoding problem in the Hamming metric has been investigated for several decades. The most effective known attacks belong to the family of Information Set Decoding algorithms, starting with Prange's method from 1962 [35] and later improvements due to Stern [40] and Dumer [13]. More recent analyses [6, 32, 33] indicate an asymptotic complexity of the form $2^{c\omega(1+\text{negl}(1))}$ for some constant c . In the regime $\omega = \text{negl}(n)$, a more refined study confirms this estimate and makes explicit the dependence of c on the code rate k/n [12]. For the ISD algorithm known, the complexity of the classical attack is 2^{λ} , in which λ refers to the security level bits. Using Grover algorithm, the complexity of the quantum attack can be approximately quadratic, as well as $2^{\lambda/2}$. It should be noticed that ISD attack cannot be applied in solving EPC-P problem.

Specific structural attacks. Because quasi-cyclic codes possess additional algebraic structure, they may be subject to attacks that do not apply to arbitrary linear codes. One generic example is the DOOM attack [38], which, due to cyclicity, gives a gain of $O(\sqrt{n})$; for MDPC codes, the corresponding gain is $O(n)$ because the code is generated by a basis of low-weight vectors. Another possible line of attack exploits the factorization pattern of the polynomial defining the cyclic structure. Such attacks were studied in [23, 29, 38], and they become particularly effective when $x^n - 1$ has several low-degree factors. They lose their efficiency when $x^n - 1$ has only two irreducible factors, namely $(x - 1)$ and $x^{n-1} + x^{n-2} + \dots + x + 1$. This situation occurs when n is prime and q is a generator of the multiplicative group $(\mathbb{Z}/n\mathbb{Z})^*$. Such values of n are known for very large parameters, and our parameter choices use

such primitive values of n . For the structural attack, the complexity of the classical attack is 2^λ as well as the complexity of the quantum attack, in which λ refers to the security level.

Code-equivalence attacks. Another natural attack is to regard the EPC- P problem as a variant of the code equivalence problem. In this setting, the attacker tries to recover the hidden equivalence transformation and the hidden puncturing map. Classical code-equivalence techniques, such as support-splitting type algorithms, may be effective when enough structural invariants of the code are preserved. However, in the punctured setting, the attacker must also determine which positions have been removed, which adds an additional combinatorial difficulty. Thus, compared with ordinary code equivalence, the punctured version is expected to be harder for the proposed parameters. For the code-equivalence attack, the complexity of the classical attack is 2^λ as well as the complexity of the quantum attack, in which λ refers to the security level.

Security of the EPC-P problem. For the EPC-P problem, the security of the first bound relies on the hardness of distinguishing the structured public-key component $\mathbf{G}'$ from a uniformly random binary matrix. More precisely, in the reduction, the only modified component is the distribution of $\mathbf{G}'$, while the remaining parts of the public key and the experiment are unchanged. Therefore, any adversary that distinguishes the corresponding two experiments can be converted into a distinguisher for the EPC-P problem.

Security of the 2-DQCSD-P and 3-DQCSD-P problems. For the 2-DQCSD-P problem, the reduction to the 2-DQCSD problem loses one bit of security. For the 3-DQCSD-P problem, the use of `ExpandVect` in the ciphertext-related component does not introduce any additional security loss. Indeed, `ExpandVect` only pads the vector with zero coordinates in the higher positions, and this operation is public and deterministic. In both the structured and the random cases, the same zero-padding map is applied after the underlying vector has been sampled. Therefore, `ExpandVect` does not add independent information to the distribution. Any distinguisher for the expanded distributions can be converted into a distinguisher for the original 3-DQCSD-P distributions by first applying the same padding map to its input.

7. Statement of Algorithm Characteristics

This section presents an overview of the main characteristics of the HEP-QC algorithm. The discussion is intended to provide a clear understanding of the algorithm’s design features, implementation properties, and security characteristics.

7.1 Novelty

The primary innovation of HEP-QC is the introduction of an additional security layer based on the Equivalent Punctured Code problem (EPC-P). Unlike the original HQC construction, which relies predominantly on quasi-cyclic syndrome decoding (QCSD) assumptions, HEP-QC provides two independent security reductions: one to the EPC-P problem and another to the 2-DQCSD-P and 3-DQCSD-P problems. This strengthens the structural hiding property of the public key by concealing the structured component under a code-equivalence-type assumption, thereby offering a more robust security justification.

7.2 Flexibility

HEP-QC offers high flexibility through multiple parameter sets designed to meet different security levels. As shown in Table 4.1, the specification includes instances for security levels 1, 3, 5, and 7 (HEP-QC-1, HEP-QC-3, HEP-QC-5, HEP-QC-7). The parameters, including the lengths of the Reed-Solomon code, the Reed-Muller code, the ambient space, and the weights of various error vectors, can be adjusted to achieve different trade-offs between security, key sizes, and performance. The use of concatenated codes allows for internal adjustments to the error-correcting capability.

7.3 Extensibility

The framework is extensible. The structure of this framework allows for future substitution of the internal or external codes with other linear codes without altering the core HEP-QC framework. Furthermore, the scheme is adaptable to other metrics or code families.

References

- [1] Carlos Aguilar-Melchor, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, and Gilles Zémor. Efficient encryption from random quasi-cyclic codes. *IEEE Transactions on Information Theory*, 64(5):3927–3943, 2018.
- [2] Francesco Antognazza, Alessandro Barenghi, Gerardo Pelosi, and Ruggero Susella. A High Efficiency Hardware Design for the Post-Quantum KEM HQC. In *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 431–441. IEEE, 2024.
- [3] Elwyn Berlekamp. *Algebraic coding theory*. World Scientific, 1968.
- [4] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. Cryptography with constant input locality. In Alfred Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 92–110. Springer, Berlin, Heidelberg, August 2007.
- [5] Nicolas Aragon, Philippe Gaborit, and Gilles Zémor. HQC-RMRS, an instantiation of the HQC encryption framework with a more efficient auxiliary error-correcting code. <https://arxiv.org/abs/2005.10741>.
- [6] Anja Becker, Antoine Joux, Alexander May, and Alexander Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 520–536. Springer, Berlin, Heidelberg, April 2012.
- [7] Elwyn Berlekamp. *Algebraic coding theory*. World Scientific, 1968.
- [8] Elwyn R Berlekamp, Robert J McEliece, and Henk CA van Tilborg. On the inherent intractability of certain coding problems. *IEEE Transactions on Information Theory*, 24(3):384–386, 1978. <http://authors.library.caltech.edu/5607/1/BERieeetit78.pdf>.
- [9] Daniel J Bernstein. Grover vs. McEliece. In *Post-Quantum Cryptography*, pages 73–80. Springer, 2010. <https://cr.yp.to/codes/grovercode-20091123.pdf>.
- [10] Daniel J Bernstein, Tanja Lange, and Christiane Peters. Attacking and defending the McEliece cryptosystem. In *Post-Quantum Cryptography*, pages 31–46. Springer, 2008. <https://cr.yp.to/codes/mceliece-20080807.pdf>.
- [11] Anne Canteaut and Florent Chabaud. A new algorithm for finding minimum weight words in a linear code: application to McEliece cryptosystem and to narrow-sense BCH codes of length 511. *IEEE Transactions on Information Theory*, 44(1):367–378, 1998. <http://ieeexplore.ieee.org/document/651067/>.
- [12] Rodolfo Canto Torres and Nicolas Sendrier. Analysis of information set decoding for a sub-linear error weight. In Tsuyoshi Takagi, editor, *Post-Quantum Cryptography - 7th International Workshop, PQCrypto 2016, Fukuoka, Japan, February 24–26, 2016, Proceedings*, volume 9606 of *Lecture Notes in Computer Science*, pages 144–161. Springer, 2016. <https://hal.inria.fr/hal-01244886>.
- [13] Ilya Dumer. On minimum distance decoding of linear codes. In *Proc. 5th Joint Soviet-Swedish Int. Workshop Inform. Theory*, pages 50–52, 1991. https://www.researchgate.net/publication/296573348_On_minimum_distance_decoding_of_linear_codes.

- [14] Matthieu Finiasz and Nicolas Sendrier. Security bounds for the design of code-based cryptosystems. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 88–105. Springer, Berlin, Heidelberg, December 2009.
- [15] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999.
- [16] Philippe Gaborit. Shorter keys for code based cryptography. In *Proceedings of the 2005 International Workshop on Coding and Cryptography (WCC 2005)*, pages 81–91, 2005. http://www.unilim.fr/pages_perso/philippe.gaborit/shortIC.ps.
- [17] Philippe Gaborit and Marc Girault. Lightweight code-based identification and signature. In *2007 IEEE International Symposium on Information Theory*, pages 191–195. IEEE, 2007. https://www.unilim.fr/pages_perso/philippe.gaborit/isit_short_rev.pdf.
- [18] Shuhong Gao and Todd Mateer. Additive Fast Fourier Transforms over finite fields. *IEEE Transactions on Information Theory*, 56(12):6265–6272, 2010.
- [19] Suhong Gao and Todd Mateer. Additive fast Fourier transforms over finite fields. *IEEE Transactions on Information Theory*, 56(12):6265–6272, 2010.
- [20] Lewis Glabush, Kathrin Hövelmanns, and Douglas Stebila. Tight Multi-challenge Security Reductions for Key Encapsulation Mechanisms. *Cryptology ePrint Archive*, Paper 2025/343, 2025. <https://eprint.iacr.org/2025/343>.
- [21] Danilo Gligoroski. PQC forum, official comment on BIKE submission. NIST PQC forum, December 2017. <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/round-1/official-comments/BIKE-official-comment.pdf>.
- [22] Qian Guo, Clemens Hlauschek, Thomas Johansson, Norman Lahr, Alexander Nilsson, and Robin Leander Schröder. Don’t reject this: Key-recovery timing attacks due to rejection-sampling in HQC and BIKE. *IACR TCHES*, 2022(3):223–263, 2022.
- [23] Qian Guo, Thomas Johansson, and Carl Löndahl. A new algorithm for solving Ring-LPN with a reducible polynomial. *IEEE Transactions on Information Theory*, 61(11):6204–6212, 2015. <https://arxiv.org/abs/1409.0472>.
- [24] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In *Theory of Cryptography Conference*, pages 341–371. Springer, 2017.
- [25] W. Cary Huffman and Vera Pless. *Fundamentals of error-correcting codes*. Cambridge university press, 2010. <https://www.amazon.fr/Fundamentals-Error-Correcting-Codes-Cary-Huffman/dp/0521131707>.
- [26] Shu Lin and Daniel J Costello. *Error control coding*, volume 2. Prentice Hall Englewood Cliffs, 2004.
- [27] Zhen Liu and Yanbin Pan. PQC forum, official comment on HQC submission. NIST PQC forum, January 2018. <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/round-1/official-comments/HQC-official-comment.pdf>.
- [28] Zhen Liu, Yanbin Pan, and Tianyuan Xie. Breaking the hardness assumption and IND-CPA security of HQC submitted to NIST PQC project. In *International Conference on Cryptology and Network Security*, pages 344–356. Springer, 2018.

- [29] Carl Löndahl, Thomas Johansson, Masoumeh Koochak Shooshtari, Mahmoud Ahmadian-Attari, and Mohammad Reza Aref. Squaring attacks on McEliece public-key cryptosystems using quasi-cyclic codes of even dimension. *Designs, Codes and Cryptography*, 80(2):359–377, 2016. <https://link.springer.com/article/10.1007/s10623-015-0099-x>.
- [30] Florence Jessie MacWilliams and Neil James Alexander Sloane. The theory of error-correcting codes, volume 16. Elsevier, 1977.
- [31] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error-correcting codes*, volume 16. Elsevier, 1977.
- [32] Alexander May, Alexander Meurer, and Enrico Thomae. Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In *Asiacrypt*, volume 7073, pages 107–124. Springer, 2011. https://link.springer.com/chapter/10.1007/978-3-642-25385-0_6.
- [33] Alexander May and Ilya Ozerov. On computing nearest neighbors with applications to decoding of binary linear codes. In *EUROCRYPT (1)*, pages 203–228, 2015. <http://www.cits.rub.de/imperia/md/content/may/paper/codes.pdf>.
- [34] Rafael Misoczki, Jean-Pierre Tillich, Nicolas Sendrier, and Paulo SLM Barreto. MDPC-McEliece: New McEliece variants from moderate density parity-check codes. In *Information Theory Proceedings (ISIT), 2013 IEEE International Symposium on*, pages 2069–2073. IEEE, 2013. <https://eprint.iacr.org/2012/409.pdf>.
- [35] Ray Perlner. Security strength categories for Code Based Crypto. Public comment on NIST pqc-forum mailing list, 2021.
- [36] Eugene Prange. The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory*, 8(5):5–9, 1962. <http://ieeexplore.ieee.org/document/1057777/>.
- [37] Markku-Juhani O. Saarinen. IND-CCA2 issue in HQC. Public comment on NIST pqc-forum mailing list, 2025.
- [38] Nicolas Sendrier. Decoding one out of many. In *International Workshop on Post-Quantum Cryptography*, pages 51–67. Springer, 2011. <https://eprint.iacr.org/2011/367.pdf>.
- [39] Nicolas Sendrier. Secure Sampling of Constant Weight Words – Application to BIKE. Cryptology ePrint Archive, Report 2021/1631, 2021. <https://eprint.iacr.org/2021/1631>.
- [40] Jacques Stern. A method for finding codewords of small weight. In *International Colloquium on Coding Theory and Applications*, pages 106–113. Springer, 1988. <https://link.springer.com/chapter/10.1007/BFb0019850>.
- [41] C. Wieschebrink. Two NP-complete problems in coding theory with an application in code based cryptography, In *2006 IEEE International Symposium on Information Theory, Seattle*, pages. 1733–1737, IEEE, 2006. <https://ieeexplore.ieee.org/abstract/document/4036264>